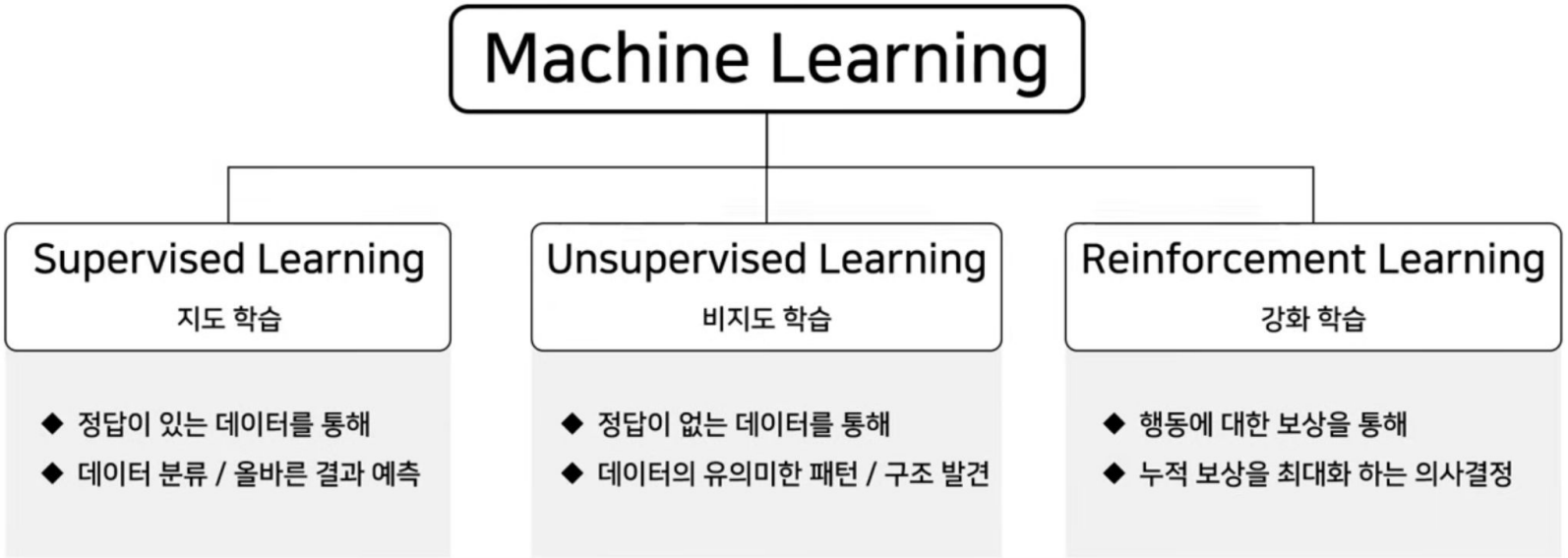
# 머신러닝 실습

## 시나리오 scenario 와 알고리즘 algorithm

# 머신러닝 Machine Learning

## 머신러닝

컴퓨터 알고리즘에 대량의 데이터를 입력하여 데이터 간의 관계 및 패턴을 스스로 학습하고, 이를 바탕으로 새로운 데이터에 대한 예측이나 분류 및 추천을 하는 인공지능 AI 의 하위 분야



# 지도 학습 Supervised Learning

## 지도 학습 Supervised Learning

**정답(Label)이 포함된 훈련 데이터를 사용**하여 인공지능 모델을 학습시키는 머신러닝 방식입니다. 입력 데이터와 그에 대응하는 출력값의 관계를 파악하여 새로운 데이터가 들어왔을 때 결과를 예측하는 것이 목표이며 **분류(Classification)와 회귀(Regression) 문제 해결**에 주로 활용

## 알고리즘

1. KNN Classification
2. KNN Regression
3. (Simple) Linear Regression
4. Multiple Linear Regression
5. Polynomial Linear Regression

# 시나리오 #1

우리 온라인커머스 회사는 그동안 직원들이 택배포장을 하고 무게와 크기에 따라 적절한 **택배요금 구간(소형,중형,대형)을 분류**해 왔습니다.

근데 이제 AI 시대가 되었으니 우리도 첨단회사에 맞게 자동화시스템을 도입하겠다고 생각했습니다.
(담당 직원들을 해고시켜서 인건비를 아끼고 싶기도 하지만 비밀입니다.)

이 분류 자동화는 아주 간단한 계산으로도 가능해 보이네요. 특정 무게와 길이에 따라 구분하면 되니 조건 IF문을 사용하면 간단하게 구현이 가능합니다. 그래서 야심차게 개발자를 통해 무게와 크기에 따른 자동 분류시스템을 만들어 도입했습니다. (분류 업무 직원도 모두 해고하고요..ㅜㅜ)

python 코드----------

```
if 무게<3 and 크기<30:
    return '소형'
elif 무게<10 and 크기<100:
    return '중형'
else
    return '대형'
```

---

근데. 도입후 사용자들로부터 택배요금에 대해 수많은 컴플레인을 받게 되었습니다.
왜지? 특별하게 코드에는 문제가 없었는데 말이죠.

# 시나리오 #1

사례를 살펴봅니다.

3.02kg 으로 '중형' ← *"아니 그전까지는 같은 제품이 '소형'이었는데 **0.02kg**으로 '중형'!!!"*
101cm 로 '대형' ← *"고작 **1cm** 오버라고.. 대형이라니 너무 치사해!!!!"*

흠. 예전에는 포장직원들이 대략적인 무게와 크기로 분류를 했습니다.(이를 사회통념이라고도 할수 있겠네요.) 근데 프로그램을 통하니 너무 엄격하게 구분하여 경계에 있던 고객들의 불만이 폭발했네요.

사장은. 고민이네요. 기존 직원을 다시 뽑을 수도 없고요.
그래서 AI 기술로 이 문제를 해결해 보고자 합니다.

기존 직원들이 분류했던 방식에는 불만이 없었으니 그 당시의 데이터를 기반으로 분류를 하면 이 문제를 해결할 수 있을 것 같네요.

**"유사한 데이터의 특성을 보이면 같은 택배요금구간으로 분류하도록 합시다."**

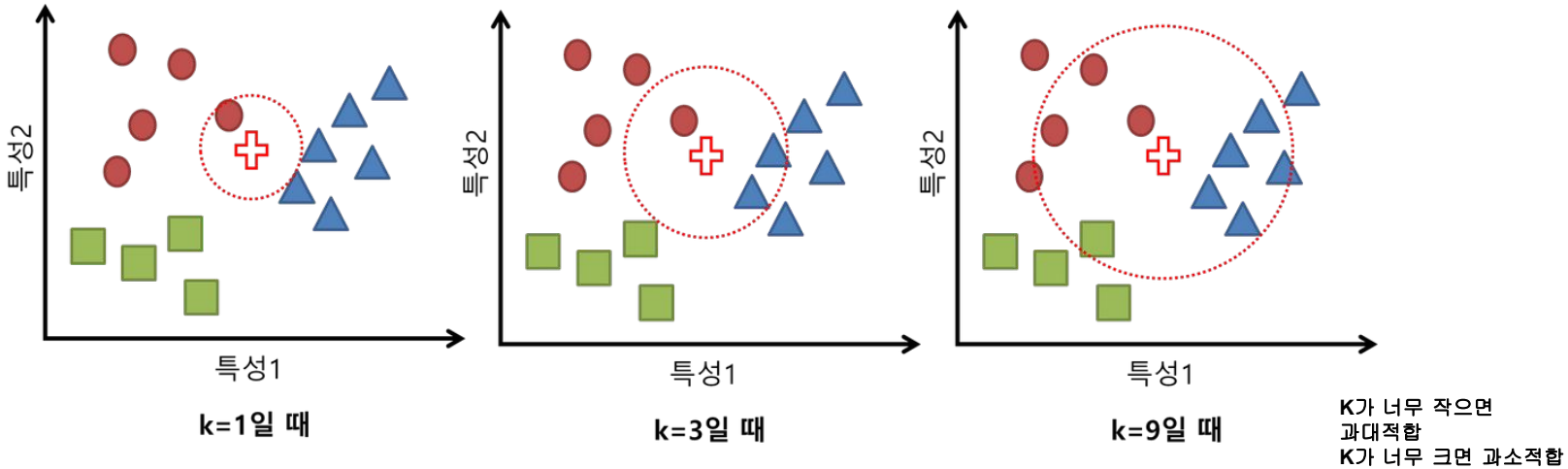
# KNN Classification

## 최근접 이웃 분류 알고리즘 K - Nearest Neighbors Classification

- 무게(kg)와 가장 긴 한변의길이(cm) 데이터를 통한 택배요금 구간 분류(소형,중형,대형)

### 알고리즘

새로운 데이터의 클래스(종류)를 주변의 가장 가까운 K 개 데이터의 다수결 투표로 분류하는 단순하고 직관적인 지도학습 알고리즘

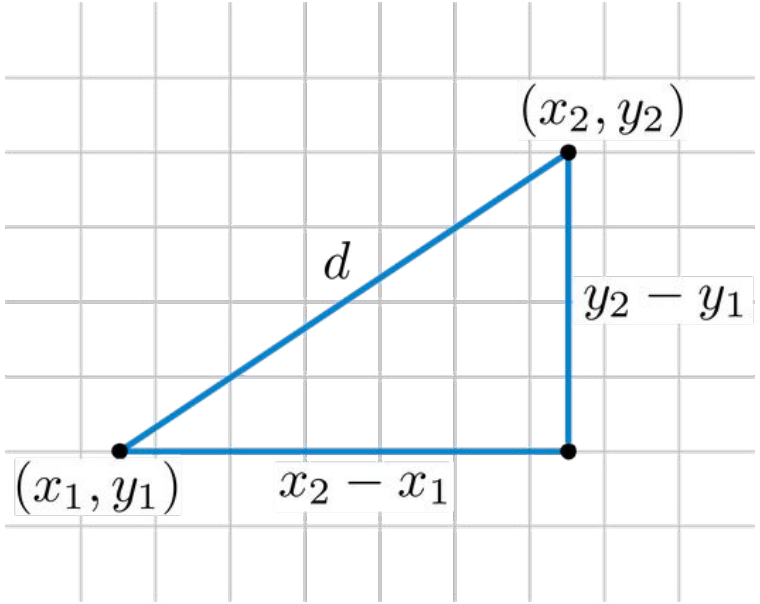



K가 너무 작으면
과대적합
K가 너무 크면 과소적합

# 거리측정 방법1.

유클리디안 거리 **Euclidean Distance**

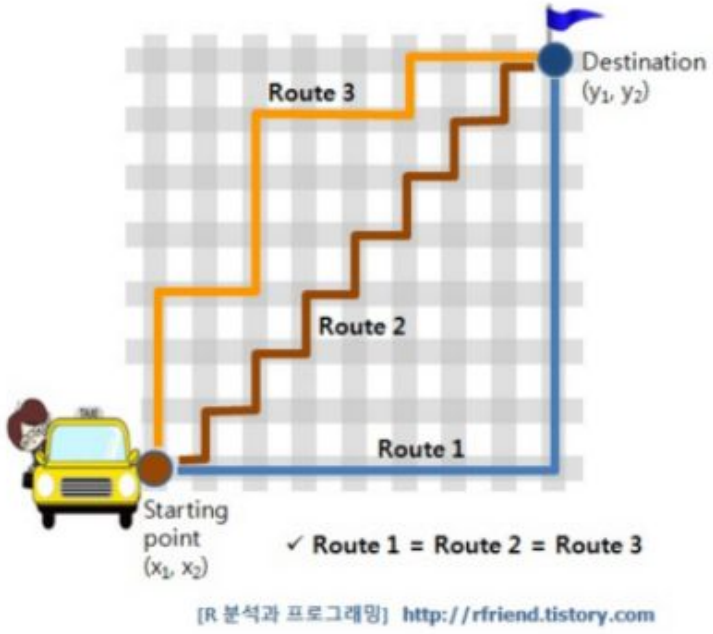
$$d(A,B) = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$



# 거리측정 방법2.

맨하탄 거리 **Manhattan Distance**

$$d(A,B) = |x1 - x2| + |y1 - y2|$$




제곱근 계산이 없는 절댓값의 합이기 때문에,
고차원 데이터셋에서 유클리드 거리보다 계산 속도가 더 빠를 수 있음

# 시나리오 #2

KNN 분류 모델을 도입한 후, 대부분의 택배는 잘 분류되고 있었습니다. 그런데 몇몇 상품에서 납득하기 어려운 결과가 나옵니다.

포장팀 직원이 말합니다.
“이 상품은 무게도 비슷하고 크기도 비슷한데 왜 전혀 다른 요금 구간으로 분류됐죠?”

머신러닝팀이 데이터를 들여다봅니다.
“이상하네요... 가장 가까운 이웃들이 엉뚱한 상품들입니다.”
“거리 계산을 보니까 무게보다는 크기(cm) 가 거의 모든 택배요금 구간을 결정하고 있 $거리 = \sqrt{무게^2 + 크기^2}$

“무게는 kg 단위이며 1~20 정도인데, 크기는 cm 단위이고 20~200이라 거리 계산에서 크기가 더 큰 숫자로 계산되어 무게는 거의 무시되고 있습니다.”
“그럼 이건 크기만 보고 분류하는 거랑 뭐가 다르죠?”

**해결 아이디어**
“무게와 크기를 같은 스케일로 맞추지 않으면 ‘가깝다’는 의미 자체가 왜곡됩니다.”

두 변수의 단위를 맞추는 대표적인 2가지 방법이 있음.
**정규화 normalization, 표준화 standardization**

이렇게 모델 학습전에 데이터의 값을 조정하는 작업을 **[데이터 전처리]** 라고 부릅니다.

# 데이터 전처리 (특성 스케일링 : Feature Scaling)

머신러닝에서 **변수(특성)의 크기 범위를 맞춰주는 작업** 을 의미

거리 기반 알고리즘(예: K-means clustering, k-nearest neighbors algorithm)이나 경사하강법을 사용하는 모델에서 매우 중요

## 정규화 (Normalization)

최대-최소 스케일링 Min-Max Scaling [ 모든 값들을 0 ~ 1 범위로 통일 ] : 총 범위안에서 내 값이 어느정도 위치에 있는가

※ 이미지 픽셀값 (0~255 → 0~1)

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

## 표준화 (Standardization)

표준점수 Standard Score (Z - Score) [ 평균이 0, 표준편차만큼이 1. ] : 내 점수가 표준편차의 몇배정도 값을 가지는 가

※ 이상치에 정규화보다 덜 민감

※ 대부분의 머신러닝 모델에서 적합

$$z = \frac{x - \mu}{\sigma}$$

# 시나리오 #3

KNN 알고리즘 도입으로 고객 불만은 해결되었네요.

근데 매출이 계속 정체되어 뭔가 마케팅을 해야 할 것 같습니다.
근데 어느정도의 광고비를 책정해야 할지 막막하네요. 그래서 기존 광고비에 따른 매출액 데이터를 통해 얼마의 광고비를 태울지 결정해 보려 합니다.

이전에 분류에 했던 KNN을 조금 활용하면 예측도 가능할 것 같아요.

**가까운 광고비에 따른 매출액의 평균**을 주면 어떨까요?
이게 **KNN Regression(회귀) 알고리즘** 입니다.

분류가 '무엇인가'를 맞히는 것이라면, **회귀(Regression)**는 임의의 **숫자(수치)**를 예측하는 것

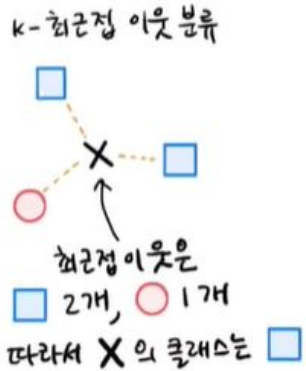
# KNN Regression

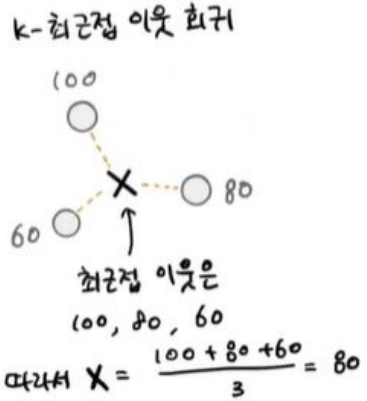
## 최근접 이웃 회귀 알고리즘 K - Nearest Neighbors Regression

- 광고비(원) 에 따른 매출액(원) 예측

### 알고리즘

예측하려는 샘플에서 가장 가까운 K 개의 이웃을 찾고, 그 이웃들의 결과값을 **산술 평균** 내어 예측값으로 사용하는 단순하고 직관적인 지도학습 알고리즘



# 모델의 성능평가

## MAE (Mean Absolute Error, 평균절대오차 )

**정의**: 실제값과 예측값 차이의 절댓값 평균

**해석**: 예측이 평균적으로 얼마나 틀렸는지

**특징**: 이상치(outlier)에 **덜 민감, 작을수록 좋은**

$$MAE = \frac{1}{n}\sum|y - \hat{y}|$$

## MSE (Mean Squared Error, 평균제곱오차 )

**정의**: 오차를 제곱한 값의 평균

**특징**: 오차를 제곱하기 때문에 **큰 오차에 매우 민감**

단위가 제곱 단위라 직관성이 떨어질 수 있음

$$MSE = \frac{1}{n}\sum(y - \hat{y})^2$$

## RMSE (Root Mean Squared Error, 평균제곱근오차 )

**정의**: MSE에 루트를 씌운 값

**특징**: 실제 데이터와 **같은 단위**

큰 오차에 민감 (MSE와 동일한 특성)

$$RMSE = \sqrt{MSE}$$

# 모델의 성능평가

## $R^2$ (결정계수 , Coefficient of Determination)

**범위**: 보통 **0 ~ 1 ( 1에 가까울수록 좋음 )**

**해석**: 모델이 데이터 변동성을 얼마나 설명하는지 비율

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

**$R^2$ = 1 − (모델 오차 ÷ 평균 편차)**

모델 오차가 작아질수록 → 분수가 작아짐 → **$R^2$는 1에 가까워짐**

모델이 평균 예측과 같으면 → **$R^2$ = 0**

평균보다 못하면 → **$R^2$ < 0**

완벽 예측이면 → **$R^2$ = 1**

scikit-learn 에서 회귀모델의 `.score()`는 기본적으로 **$R^2$(결정계수 )** 를 반환

직관적인 해석 (설명력) : 1에 가까울수록 좋음

스케일에 독립적 : MAE, MSE, RMSE는 **데이터 단위에 따라 값이 달라짐**

$R^2$는 **비율값** → 단위 영향 없음

$$SS_{res} = \sum (y - \hat{y})^2$$

**Residual Sum of Squares (잔차제곱합)**

실제값과 예측값 차이의 제곱합

$$SS_{tot} = \sum (y - \bar{y})^2$$

**Total Sum of Squares (총제곱합)**

실제값과 평균값 차이의 제곱합

# 시나리오 #4

KNN Regression 을 적용하여, 목표로 하는 매출액을 달성하기 위해 광고비 예산을 적절하게 집행하는 것이 가능해 졌네요.

가만 보니 광고비가 증가하면 매출액이 그 만큼 상승하고 있네요. 광고비를 추가로 집행하여 회사의 매출을 키워보고자 합니다.
기존 최고 광고비의 2배에 달하는 광고비를 투자하면 얼마나 매출액이 나올지 예측해보고자 합니다.

그래서 기존 KNN Regression 모델에 2배의 금액을 넣었는데. 어처구니 없이 예상 매출액이 기존 최고액과 차이가 별로 없어요.

왜죠?

아. 알고리즘의 특성상 가까운 기존 데이터들의 평균값을 결과값으로 사용하기에 그 이상의 값을 예측하는 것은 불가능합니다.

다른 알고리즘을 사용해야 겠군요.
가만보니, 광고비에 따라 매출액의 값들이 우 상향 추세를 보이고 있네요.
그 우상향 추세선을 더 길게 그려보면 광고비에 따른 매출액을 구할 수 있지 않을까 싶네요.

이 **직선의 추세선을 통해 예측하는 알고리즘을 "선형회귀 Linear Regression"**이라고 합니다.

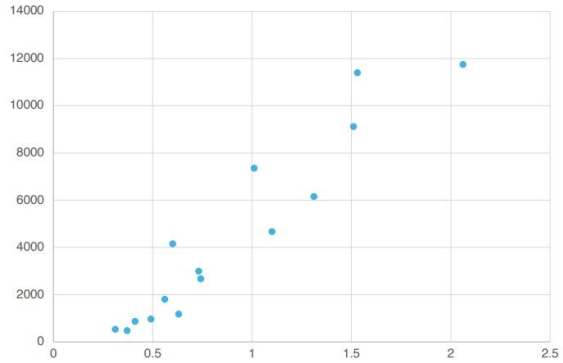
# Linear Regression

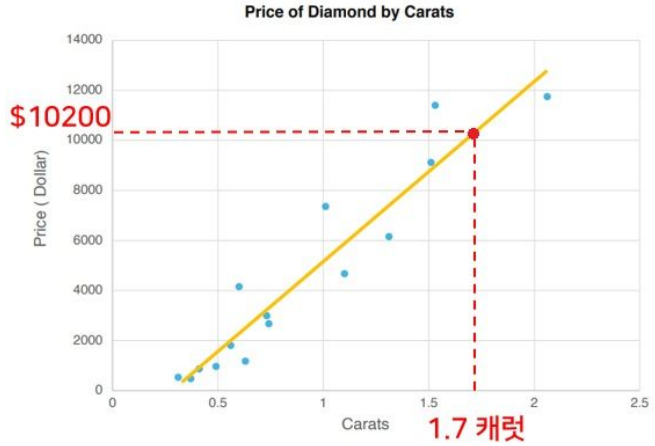
## 선형 회귀 알고리즘 (단순 Simple 선형회귀 - 독립변수 1개)

- 광고비(원) 에 따른 매출액(원) 예측 개선

### 알고리즘

하나의 독립 변수(X)와 종속변수(Y)간의 선형 관계를 모델링하는 지도학습 알고리즘.
데이터를 가장 잘 대변하는 최적의 직선(회귀선)을 찾아 미래 값을 예측하는 통계 기법. 집값 예측 등 연속적인 수치 예측에 활용




$$y = wx + b$$



$$y = ax + b$$

$$f(x) = ax + b$$

$$H(x) = wx + b$$

*Hypothesis* *Weight* *Bias*

$$y = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n + b$$





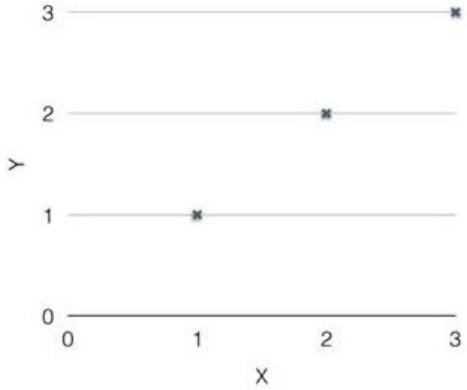




# Linear Regression

$$H(x) = wx + b$$

| x | Y |
|---|---|
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |




$$y = 1 \times x + 0$$

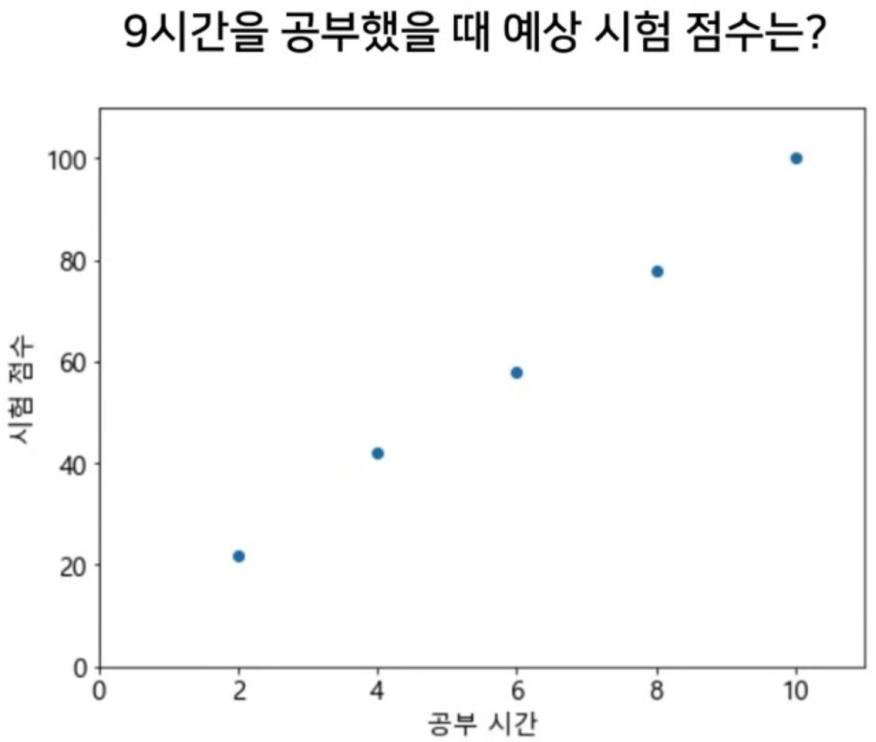
# 선형 회귀를 이용한 예측

사과개수, 가격 : (1개, 1100원), (2개, 2100원), (3개, 3100원)
이렇게 3개를 학습했다면. 5개일때는 얼마?

H(가격) = W(1000)*X(개수) + B(100)

# Linear Regression

| 공부 시간 | 시험 점수 |
|---|---|
| 2 시간 | 22 점 |
| 4 시간 | 42 점 |
| 6 시간 | 58 점 |
| 8 시간 | 78 점 |
| 10 시간 | 100 점 |



# Linear Regression

X

Independent variable
독립 변수 (원인)

| 공부 시간 | 시험 점수 |
|---|---|
| 0.5 | 10 |
| 1.2 | 8 |
| 1.8 | 14 |
| 2.4 | 26 |
| 2.6 | 22 |
| 3.2 | 30 |
| 3.9 | 42 |
| 4.4 | 48 |
| 4.5 | 38 |
| ... | ... |

# Linear Regression

| X<br>Independent variable<br>독립 변수 (원인) | y<br>Dependent variable<br>종속 변수 (결과) |
|---|---|
| 공부 시간 | 시험 점수 |
| 0.5 | 10 |
| 1.2 | 8 |
| 1.8 | 14 |
| 2.4 | 26 |
| 2.6 | 22 |
| 3.2 | 30 |
| 3.9 | 42 |
| 4.4 | 48 |
| 4.5 | 38 |
| ... | ... |

# Linear Regression

X

Independent variable
독립 변수 (원인)

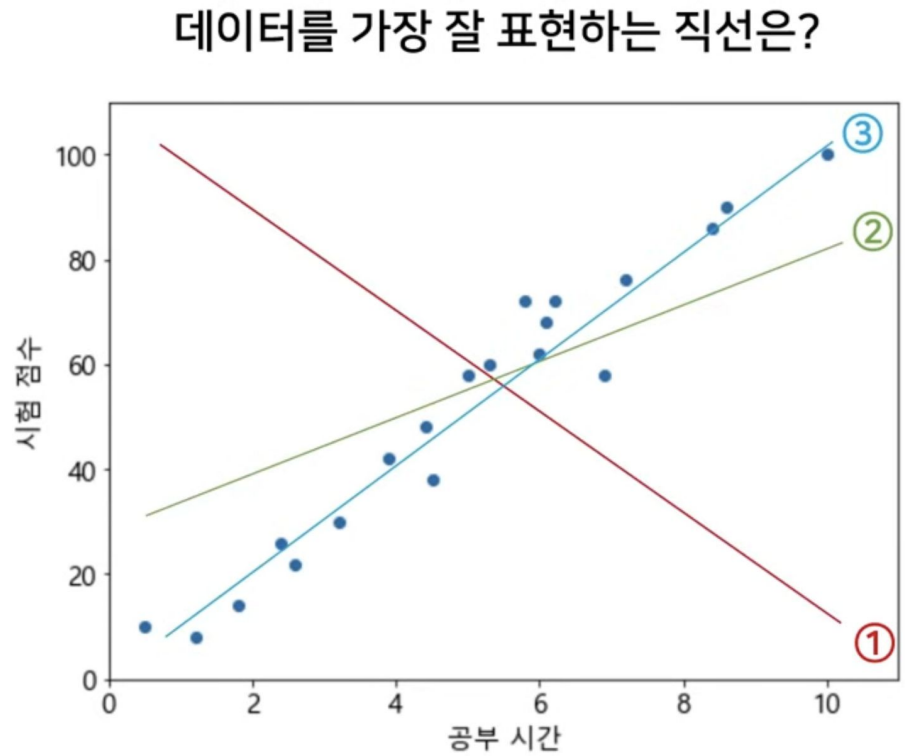
= 입력 변수, feature

y

Dependent variable
종속 변수 (결과)

= 출력 변수, target, label

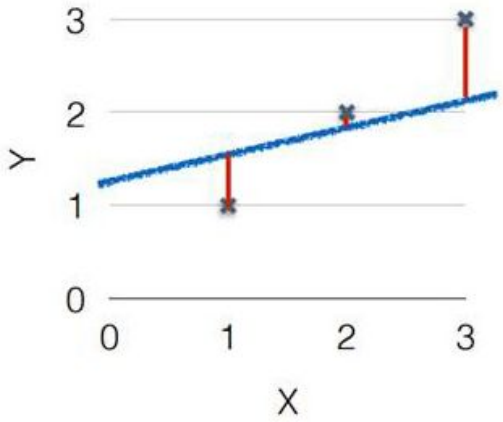
# Linear Regression





# Linear Regression

$$H(x) = wx + b$$




가설값 - 실제값

$$H(x) - y$$

$$\frac{(H(x^{(1)}) - y^{(1)})^2 + (H(x^{(2)}) - y^{(2)})^2 + (H(x^{(3)}) - y^{(3)})^2}{3}$$

$$cost = \frac{1}{m}\sum_{i=1}^{m}(H(x^{(i)}) - y^{(i)})^2$$

# 시나리오 #5

단순 선형회귀로 광고비와 매출의 관계는 훨씬 잘 설명할 수 있게 되었습니다.

하지만 마케팅 팀에서 질문이 나옵니다.
“**특정 제품**이 같은 광고비인데도 어떤 달은 잘 팔리고, 어떤 달은 안 팔립니다.”
“광고비 말고 다른 요인들도 같이 고려해야 하지 않을까요?”

“단순선형회귀는 광고비 하나로 매출을 설명하지만,
**실제 매출은 가격,시즌, 유통 채널 등 여러 요인의 선형 결합** 결과다.”

여러 요인(feature, 독립변수)에 의한 예측을 위하여 **다중 선형회귀 알고리즘** 알아보기.

# Multiple Linear Regression

## 다중 선형 회귀 알고리즘

- 광고비, 가격, 시즌(계절), 유통채널(자사몰, 오픈마켓, 라이브 커머스) 에 따른 **특정 제품 매출액 예측**

### 알고리즘

2개 이상의 독립 변수(X)를 이용하여 하나의 연속형 종속변수(Y)를 예측하거나 독립변수가 종속변수에 미치는 영향력을 분석하는 머신러닝  지도학습 알고리즘. 단순 선형회귀를 확장한 것으로여러 요인을 동시에 고려해 예측의 정확도를 높일 수 있음.

| 광고비(x1) | 가격(x2) | 시즌(x3) | 유통채널(x4) | 매출액(y) |
|---|---|---|---|---|
| 5000000 | 32000 | winter | D2C | 2100000000 |
| 5500000 | 32500 | winter | open market | 2300000000 |
| 6000000 | 33000 | spring | live commerce | 2250000000 |
| 8000000 | 29000 | summer | open market | 3800000000 |

※ 자사몰(D2C- 삼성닷컴, 나이키 공식몰), 오픈마켓(쿠팡,G마켓), 라이브커머스(인스타쇼핑라이브)

$$y = w_1x_1 + w_2x_2 + \cdots + w_ix_i + b$$

# 간경화 or 고혈압 예측 머신러닝 원리

요인) 식습관, 술, 담배, 스트레스, 수면, 유전 — 이런 요인들에 의한 질환 확률 예측.

요인(x값)이 6가지이므로

6개 중 1개의 요인을 가진 사람은 1/6 --> (0.15) 15%

6개 중 5개의 요인을 가진 사람은 5/6 --> (0.82) 82%

이런식으로 예측 가능

다만, 각 요인마다 더 강하게 영향을 미치는 요인에 가중치 값 필요. weight

이 값을 어떤값을 줄 것인가가 가장 어려운 부분임.

이를 머신러닝 알고리즘으로 예측



# 영향요인($x$)이 많을 때에는

$$H(x_1, x_2, x_3) = x_1w_1 + x_2w_2 + x_3w_3$$

| $x_1$ | $x_2$ | $x_3$ | Y |
|---|---|---|---|
| 73 | 80 | 75 | 152 |
| 93 | 88 | 93 | 185 |
| 89 | 91 | 90 | 180 |
| 96 | 98 | 100 | 196 |
| 73 | 66 | 70 | 142 |

Test Scores for General Psychology

$$\begin{pmatrix} x_1 & x_2 & x_3 \end{pmatrix} \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = \begin{pmatrix} x_1w_1 + x_2w_2 + x_3w_3 \end{pmatrix}$$

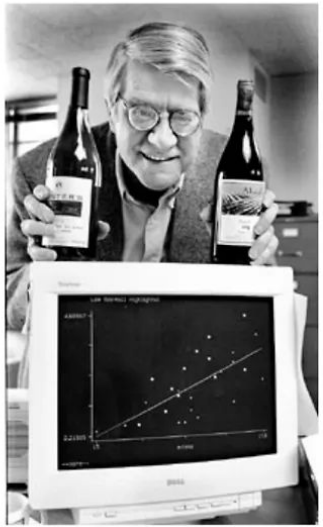
$$H(X) = XW + b$$

# Multiple Linear Regression 사례

## Supervised Machine Learning

### Supervised Machine Learning (지도 학습)

모형이 투명한 경우(White Box), 설명·예측 둘 다 활용 가능



**선형회귀 알고리즘**

$Y = a + b X_1 + c X_2 + d X_3$

**Ashenfelter's Wine Formula**

Price = 12 + 0.001 겨울강수량 + 0.06 평균온도 – 0.004 수확철강수량

**Training Data Set**

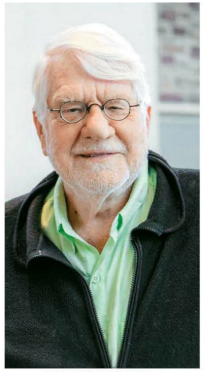
| $X_1$ 겨울 강수량 | $X_2$ 수확철 강수량 | $X_3$ 평균 온도 | Y 와인 가격 |
|---|---|---|---|
| 13 | 35 | 35 | **9.5** |
| 22 | 25 | 25 | **3.5** |
| 25 | 21 | 21 | **3.2** |
| 11 | 18 | 18 | **3.5** |
| 47 | 45 | 45 | **4.7** |
| . | . | . | . |

| 겨울 강수량 | 수확철 강수량 | 평균 온도 | 와인 가격 |
|---|---|---|---|
| 35 | 25 | 21 | ? |

ML Algo → Model → 설명

Model → 예측 12.3$

### 와인을 사랑한 두 천재의 대결

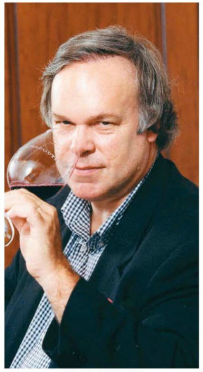


오클리 애쉔펠터 프린스턴대 교수

**방정식으로 와인에 접근한 경제학자 오클리 애쉔펠터**

"왜 내 돈 남의 혀에 맡기나"
여름 더울수록 와인은 훌륭
'보르도 방정식' 개발에 성공

1989 빈티지 훌륭 예측 적중
와인 평가의 성역 무너뜨려



로버트 파커 와인 평론가

**낭만으로서의 와인 향유 압도적 미각 로버트 파커**

와인 평론의 세계적 권위자
수많은 농장의 운명 움직여

맛도 안보고 와인 평가하는
숫자 기반 시장논리에 반발
"네안데르탈인 같은 헛소리"


© IDK² Inc. All Rights Reserved. I Don't Know What I Don't Know

# 시나리오 #6

선형회귀 모델을 도입한 뒤, 효율적인 자금운영이 가능해졌습니다.이로 인해 회사가 더욱 성장하였습니다. 덕분에 새로운 투자도 받을 수 있었습니다.
이제 회사에서 더욱 큰 도약을 준비하고 있습니다.

“광고비를 늘리면 매출도 늘어난다.” 실제로 소규모 캠페인에서는 효과가 확실했습니다.
1천만 원을 쓰면 눈에 띄게 매출이 올랐고, 2천만 원을 쓰면 더 크게 반응했습니다.

그래서 사장은 이렇게 말합니다.
“좋은데요? 그럼 이번엔 5배로 가봅시다. 한 번에 크게 밀어붙여요.”

하지만 결과는 예상과 달랐습니다.광고는 도처에 넘쳐났지만 매출은 생각만큼 늘지 않았습니다.
오히려 고객센터에는 이런 문의가 늘어났습니다.

“요즘 왜 이렇게 광고가 많죠?”
“같은 광고만 계속 나와서 불편해요.”

마케팅 팀이 회의에서 말합니다.
“처음엔 광고가 반갑지만, 너무 많아지면 사람들은 피로해집니다.”
“광고비를 두 배 쓴다고 매출이 두 배가 되지는 않아요.”

# 시나리오 #6

그제서야 사장은 깨닫습니다.
광고비와 매출의 관계는 단순한 직선이 아니구나.
어느 구간까지는 효과가 커지고, 그 이후에는 효율이 떨어지며, 때로는 오히려 역효과가 나기도 합니다. 이처럼 한 방향으로 쭉 뻗은 직선이 아니라 굽이치는 곡선에 가까운 관계를 모델링해야 할 필요가 생겼습니다. 즉, 비선형 함수의 특징을 가지고 있네요.

**“현실 세계에서는 ‘많이 쓴 만큼 항상 많이 돌아오는’ 일은 드뭅니다.”**

애석하게 기존 선형회귀모델을 선형함수 특성을 분석하기에 적합합니다.
비선형 데이터 패턴을 분석하기에는 제곱과 세제곱등 고차항을 추가하여 곡선을 학습할 수 있는 **다항회귀분석 (Polynomial Regression)**을 사용하는 것은 매우 효과적인 방법입니다.

# Polynomial Linear Regression

## 다항 선형 회귀 알고리즘

- 광고비(원) 에 따른 매출액(원)의 예측 정확도 향상

### 알고리즘

사이의 비선형 관계를 모델링하기 위해 **독립변수의 거듭제곱(차수)을 추가하여 회귀식을 곡선 형태**로 만드는 방법.
변수의 1차식만 사용하는 단순 선형회귀와 달리 데이터의 복잡한 굴절(곡선 패턴)을 학습할 수 있으며, 2차 이상의 항을 더해 다중 선형회귀의 특수한 형태로 취급 (다항회귀도 여전히 회귀계수를 사용하기에 '선형'모델로 분류됨. 즉, 선형 회귀의 확장판)

| 광고비(x) | 매출액(y) |
|---|---|
| 500 | 210000 |
| 550 | 230000 |
| 600 | 225000 |
| 800 | 380000 |

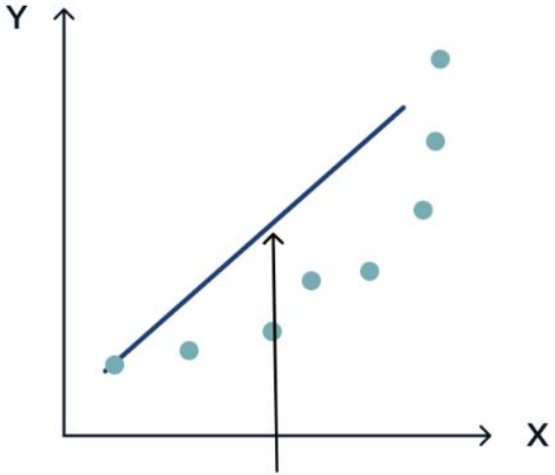
→

| 광고비 (x) | 광고비 제곱 | 광고비 세제곱 | 매출액(y) |
|---|---|---|---|
| 500 | 250000 | 125000000 | 2100000000 |
| 550 | 302500 | 166375000 | 2300000000 |
| 600 | 360000 | 216000000 | 2250000000 |
| 800 | 640000 | 512000000 | 3800000000 |

$$y = w_1 x + w_2 x^2 + \cdots + w_n x^n + b$$

# Polynomial Linear Regression

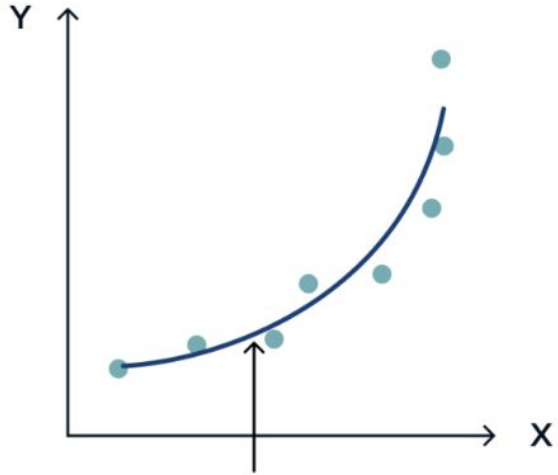
Simple linear model




$y=b_0+b_1x$

b=w

Polynomial model




$y=b_0+b_1x+b_2x_1^2$

# 시나리오 #6-1

다항회귀 모델을 도입한 뒤, 회사는 다시 한 번 성과를 냅니다.
이전보다 예측이 훨씬 정확해졌고, 과거 데이터에서는 거의 틀리지 않는 수준이었습니다.

데이터 분석팀이 보고합니다.
“이번 모델은 과거 광고 캠페인 결과를 거의 완벽하게 맞춥니다.”

사장의 얼굴이 밝아집니다.
“완벽하다면… 조금 더 정교하게 만들 수는 없을까요?(한번 실패를 했더니 무섭네요;;)”

머신러닝팀은 곡선을 더 잘 맞추기 위해 다항식의 차수를 하나 더 올립니다.

그러자 결과는 더 좋아 보였습니다.

- 과거 데이터 테스트 결과 오차: 거의 0
- 그래프: 데이터 포인트를 정확히 관통

회의실 분위기가 달아오릅니다.
“와, 이건 신의 모델 아닌가요?”

# 시나리오 **#6-1**

하지만 몇 주 뒤, 문제가 터집니다.
새로운 광고 캠페인에 이 모델을 적용했는데 예상 매출과 실제 매출이 크게 어긋난 것입니다.
어떤 캠페인은 “대박이 날 거다”라고 예측했지만 실패했고,
어떤 캠페인은 “효과가 없다”는 판단으로 기회를 놓쳤습니다.

마케팅 팀이 묻습니다.
“과거 데이터는 다 맞췄다면서요? 왜 이번엔 이렇게 틀리죠?”

데이터 분석팀이 조심스럽게 설명합니다.
“이 모델은… 과거 사례 하나하나를 외워버린 것 같습니다.”
“광고비와 매출의 일반적인 관계를 배운 게 아니라 그때그때의 특이한 상황까지 전부 공식에 끼워 맞췄습니다.”

즉, 이 모델은 시험 문제는 외웠지만, 새로운 문제는 풀지 못하는 상태였습니다.

이렇게 학습용 데이터셋(train dataset)에 너무 맞추어 학습하여 새로운 데이터에 대한 예측정확도가 떨어지는 것을 **“과대적합 overfitting”** 이라고 부름

**과대적합 문제 해결을 위한 방법 2가지 : 릿지, 라쏘**
**방법 1: 릿지(Ridge) — “다 같이 조용히 합시다. 즉, 변수들의 영향력을 조금씩 낮추자”** [Ridge 핵심 : 변수 삭제를 하지 않고, 변수 전체의 계수coefficient 를 축소]
**방법 2: 라쏘(Lasso) — “필요 없는 사람은 나갑니다, 변수 중 영향이 작은 변수는 무시(회귀계수를 0)”** [Lasso 핵심 : 중요하지 않은 변수 → 계수 0 ]

# 분류 알고리즘 Classification

- 주어진 데이터를 정해진 **범주(category)에 따라 분류** [ 예측 결과가 숫자가 아닐때]

## 알고리즘

1. KNN Classification
2. Logistic Regression
3. Decision Tree
4. Random forest ( 앙상블 )

# 시나리오 #7

우리 온라인커머스 회사는 마케팅 비용투자에 따라 매출액이 상승하는 것을 확인한 후 매달 모든 고객을 대상으로 대규모 할인 및 쿠폰 프로모션을 진행해 왔습니다.
하지만 시간이 지날수록 마케팅 비용은 늘어나는데 효과는 점점 줄어들고 있었습니다.

데이터를 분석해 보니,
이미 자주 이용하는 고객들은 별다른 혜택이 없어도 계속 서비스를 이용하고 있었습니다.
이런 고객에게 할인 쿠폰을 주는 건 **필요 없는 비용 지출**입니다. (이게 더 기분나쁘네요!!)

반대로, 최근 사용 빈도가 줄고, 접속 시간이 짧아진 고객들은 아무런 조치를 하지 않으면 조용히 서비스를 떠나는 경우가 많았습니다.

즉, 매출을 높이기 위해 회사에 중요한 것은 모든 고객이 아니라 이탈 가능성이 있는 고객을 미리 찾아내어 적절한 조치를 취해서 구매로 이어지도록 하는 것이었습니다.

**“떠날 가능성이 있는 고객만 미리 찾아내어 조치를 취할 수 없을까요?”**

그래서 회사는 한 가지 문제를 정의합니다.
“이 고객은 가까운 시일 내에 서비스를 해지할까?”

이 문제의 답은 매출액 같은 숫자가 아니라 **Yes** 또는 **No** 이다.

# 시나리오 #7

회사는 고객의 최근 행동 데이터를 활용해 이탈할 가능성을 확률로 분류하고, Yes 일 확률이 높은 고객에게만 프로모션을 제공하기로 합니다.

이처럼 특정 사건이 발생할 가능성을 예측하는 분류 문제에서 사용되는 대표적인 알고리즘이 **로지스틱 회귀(Logistic Regression)** 입니다.

AI 도입전에는 경험많은 시니어 마케터가 본인의 경험에 비추어 이런 고객이 떠날거야. 라고 통찰력을 보여주며 이 문제를 해결했지만, 해지했던 고객들의 사용패턴 데이터를 분석하면 오차를 더 줄일 수 있을 겁니다.

# Logistic Regression

## 로지스틱 회귀(분류) 알고리즘

- 최근 1개월 사용빈도, 평균 접속시간, (할인 쿠폰 사용여부 X) 등을 분석하여 해지할지 여부 분류
- 전체 회원데이터 중 서비스를 해지(회원탈퇴)한 고객들의 데이터만 추출하여 해지 1개월 전 사용패턴 분석 (탈퇴 1개월전 3가지 독립 변수 값들을 분석)

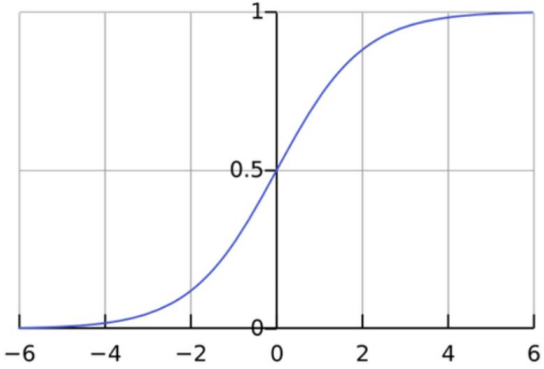
### 알고리즘

데이터가 특정 범주(0 또는 1, 예/아니오)에 속할 확률을 0~1 사이 값으로 예측하는 지도 학습 **분류** 알고리즘
선형 회귀와 달리 시그모이드(Sigmoid) 함수를 사용하여 S자 형태의 곡선으로 결과를 모델링하며주로 이진 분류(Binary Classification) 문제 해결에 활용

$$y = mx + b$$

Sigmoid Function

$$P = \frac{1}{1 + e^{-y}}$$

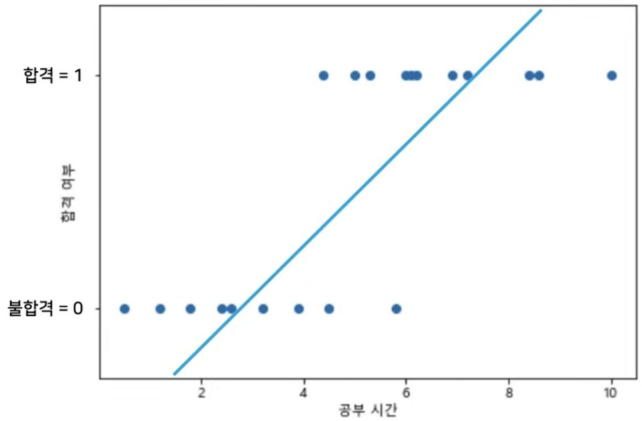


# 공부시간에 따른 합격여부 판별(분류)

| 공부 시간 | 합격 여부 |
|---|---|
| 0.5 | 불합격 |
| 1.2 | 불합격 |
| 1.8 | 불합격 |
| 2.4 | 불합격 |
| 2.6 | 불합격 |
| 3.2 | 불합격 |
| 3.9 | 불합격 |
| 4.4 | 합격 |
| 4.5 | 불합격 |
| 5 | 합격 |
| 5.3 | 합격 |
| 5.8 | 불합격 |
| 6 | 합격 |
| 6.1 | 합격 |
| 6.2 | 합격 |
| 6.9 | 합격 |
| 7.2 | 합격 |
| 8.4 | 합격 |
| 8.6 | 합격 |
| 10 | 합격 |

**“6시간 공부했을때 자격증을 딸 수 있을까?”**

선형 회귀



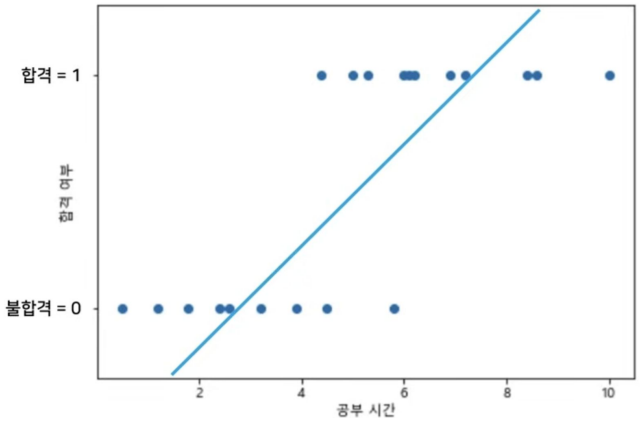

선형회귀를 사용하면 8~10시간 했을때 1을 넘는 (120%..) 어색한 확률값이 나옴

# 공부시간에 따른 합격여부 판별(분류)

| 공부 시간 | 합격 여부 |
|---|---|
| 0.5 | 불합격 |
| 1.2 | 불합격 |
| 1.8 | 불합격 |
| 2.4 | 불합격 |
| 2.6 | 불합격 |
| 3.2 | 불합격 |
| 3.9 | 불합격 |
| 4.4 | 합격 |
| 4.5 | 불합격 |
| 5 | 합격 |
| 5.3 | 합격 |
| 5.8 | 불합격 |
| 6 | 합격 |
| 6.1 | 합격 |
| 6.2 | 합격 |
| 6.9 | 합격 |
| 7.2 | 합격 |
| 8.4 | 합격 |
| 8.6 | 합격 |
| 10 | 합격 |

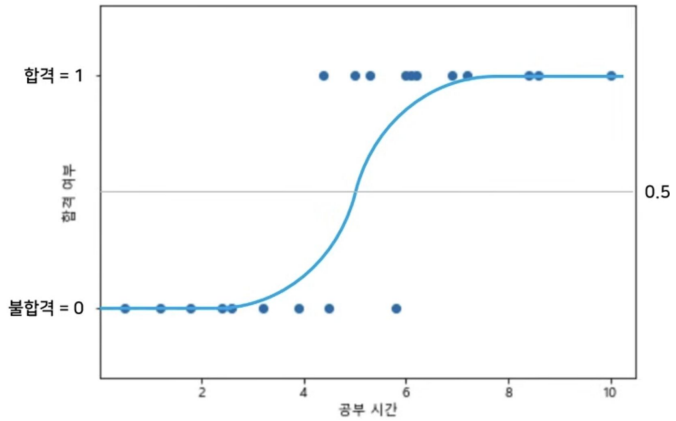
**“6시간 공부했을때 자격증을 딸 수 있을까?”**

**선형 회귀**




선형회귀를 사용하면 8~10시간 했을때 1을 넘는 (120%..) 어색한 확률값이 나옴

**로지스틱 회귀**




입력값이 아무리 커지거나 작아지더라도 0~1 범위의 값만 나오도록 계산해주는 **시그모이드(로지스틱) 함수** 사용

# 분류 모델의 성능평가 - 혼동행렬 Confusion Matrix

**분류모델의 예측결과를 실제값과 비교한 표(크로스 테이블)**

가로축 : 예측값

세로축 : 실제값

| Confusion Matrix (혼동행렬) | | Predicted (예측값) | |
|---|---|---|---|
| | | Negative | Positive |
| Actual (실제값, 정답값) | Negative | TN (True Negative) | FP (False Positive) |
| | Positive | FN (False Negative) | TP (True Positive) |

TN (True Negative) → 실제 유지 고객을 유지라고 맞춤

FP (False Positive) → 유지 고객인데 해지라고 예측 → 괜히 쿠폰 줌 (마케팅 비용 낭비 )

FN (False Negative) → 해지 고객인데 유지라고 예측 → 아무 조치 안 함 → 고객 이탈

TP (True Positive) → 실제 해지 고객을 해지라고 맞춤

이탈 예측 문제에서는: 1 = 해지 (Churn) 0 = 유지 (No Churn).

True / False(맞췄는지)

Negative / Positive (0 유지 / 1 해지 인지)

# 혼동행렬 Confusion Matrix 파생 지표

## 1. 정확도 Accuracy

전체 데이터 중 예측을 정확하게 맞춘 비율

$$\frac{TP+TN}{TP+TN+FP+FN}$$

| Confusion Matrix (혼동행렬) | | Predicted (예측값) | |
|---|---|---|---|
| | | Negative | Positive |
| Actual (실제값, 정답값) | Negative | TN (True Negative) | FP (False Positive) |
| | Positive | FN (False Negative) | TP (True Positive) |

## 2. 재현율 Recall or Sensitivity

실제 양성(positive, 1) 데이터 중 모델이 실제 양성이라고 맞춘 비율

$$\frac{TP}{TP+FN}$$

**특히 재현율이 중요 지표인 경우**

실제 Positive 양성 데이터를 Negative로 잘못 판단하게 되면
업무상 큰 영향이 발생하는 경우로,
사람의 생명이 달린 **암진단**의 경우에 중요하게 생각되는 지표

| Confusion Matrix (혼동행렬) | | Predicted (예측값) | |
|---|---|---|---|
| | | Negative | Positive |
| Actual (실제값, 정답값) | Negative | TN (True Negative) | FP (False Positive) |
| | Positive | FN (False Negative) | TP (True Positive) |

## 3. 정밀도 Precision

모델이 양성이라고 예측한 데이터 중 실제 양성인 비율

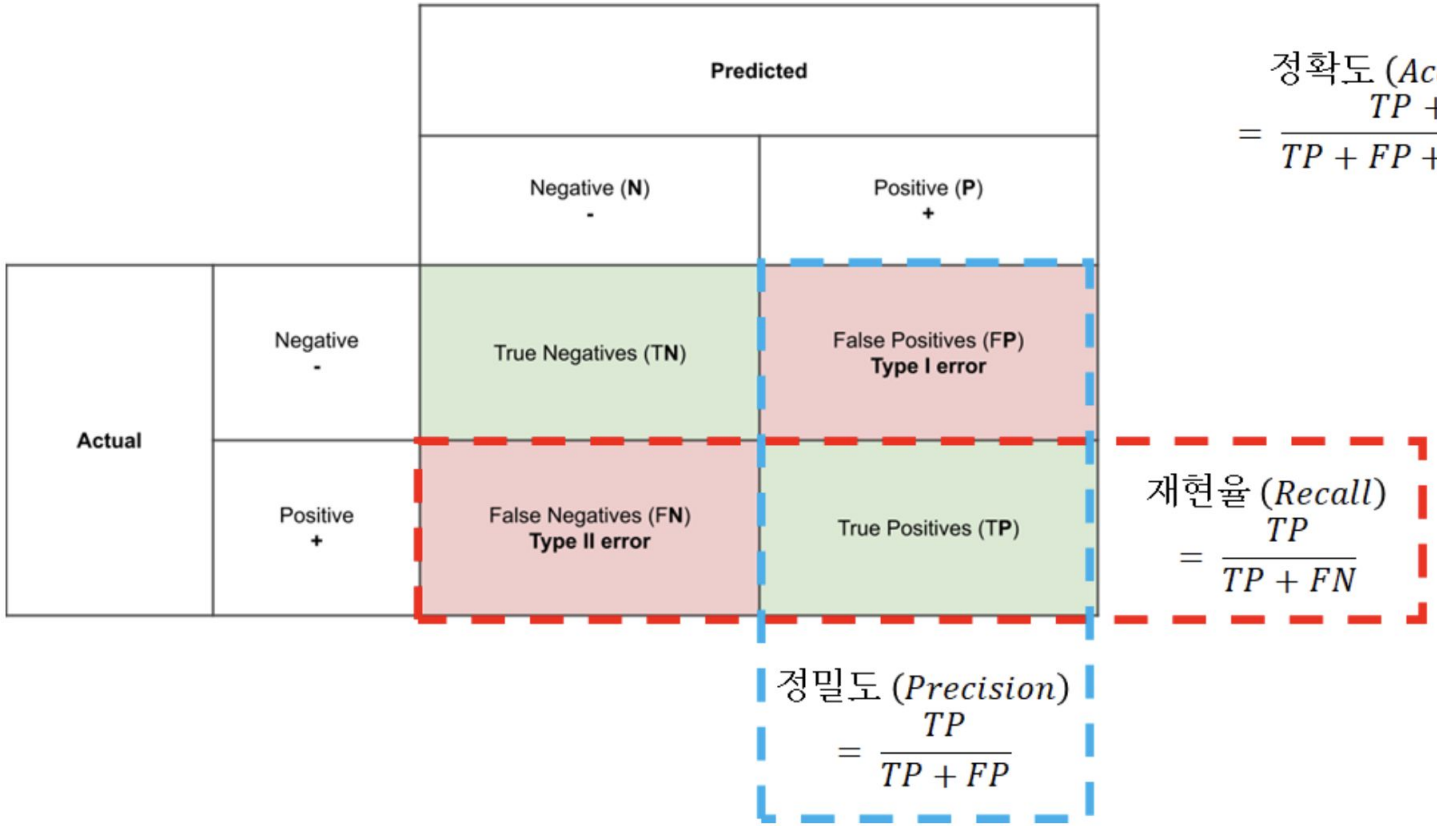
$$\frac{TP}{TP+FP}$$

**정밀도가 중요한 경우**

**스팸메일 여부를 판단하는 모델**이 대표적인 예
해당 모델의 경우 실제 Positive인 스팸 메일을 Negative인 일반 메일로 분류하더라도
사용자가 불편함을 느끼는 정도이지만,
실제 Negative인 일반 메일을 Positive인 스팸 메일로 분류할 경우 메일을 아예 받지 못하게 되어 업무에 차질이 생길 수 있음.

| Confusion Matrix (혼동행렬) | | Predicted (예측값) | |
|---|---|---|---|
| | | Negative | Positive |
| Actual (실제값, 정답값) | Negative | TN (True Negative) | FP (False Positive) |
| | Positive | FN (False Negative) | TP (True Positive) |

# 혼동행렬 Confusion Matrix 파생 지표



# 시나리오 #8

로지스틱 회귀 모델을 도입한 이후,
회사는 이탈 확률이 높은 고객들을 미리 선별하여 타겟 프로모션을 진행할 수 있게 되었습니다.
그로인해 무작위 프로모션에 비해 마케팅 비용은 줄었고 이탈률도 어느 정도 개선되는 효과를 얻었습니다.

하지만 실제 운영 과정에서 마케팅 부서로부터 새로운 피드백이 나오기 시작했습니다.
로지스틱 모델은 고객의 해지 여부를 Yes / No 혹은 확률로는 잘 예측했지만,
그 결과를 정책이나 전략으로 설명하기에는 부족했습니다.

마케터들은 이런 질문을 던졌다.
“해지 확률이 높은 건 알겠는데, 왜 위험한 고객인지 모르겠어요.”
“쿠폰을 주지 말아야 할 고객을 명확한 기준으로 설명해 주세요.”
“마케팅 정책으로 쓰려면 규칙 형태가 필요합니다.”
“고객 행동을 if–then 규칙으로 나눠서 이탈로 이어지는 패턴을 직접 보고 싶어요.”

즉, 예측 결과뿐만 아니라 판단에 이르는 과정이 필요했던 것입니다.

머신러닝팀은 새로운 알고리즘을 통해 고객 데이터를 다시 살펴보았습니다.
고객의 이탈가능성 별로 분류해보니 특정 행동 패턴이 반복적으로 나타나고 있었습니다.

# 시나리오 #8

**Rule 1: “조용히 떠나는 고객”**
IF 최근 1개월 사용빈도 ≤ 3
AND 평균 접속시간 ≤ 2분
AND 최근 3개월 쿠폰 사용 = 0
**THEN 해지 확률 높음**
“혜택에도 반응하지 않고, 서비스 이용도 거의 하지 않는 고객”

**Rule 2: “쿠폰에 반응하는 잠재 이탈 고객”**
IF 사용빈도 감소
AND 평균 접속시간 감소
AND 쿠폰 사용 경험 있음
**THEN 해지 확률 중간**
전략적 해석 : 쿠폰 제공 → 유지 가능, 쿠폰 미제공 → 이탈 가능

**Rule 3: “착한 장기 고객”**
IF 가입기간 ≥ 2년
AND 최근 사용 감소
**THEN 해지 확률 낮음**
“일시적으로 뜸해졌을 뿐, 당장 개입할 필요는 없음”

# 시나리오 #8

이런 규칙들은 고객을 하나의 기준으로 나누는 것이 아니라, 질문을 하나씩 따라가며 서로 다른 경로로 분기되는 구조를 가지고 있습니다.

이처럼 고객의 행동과 속성을 기준으로 질문을 따라 내려가며 의사결정을 수행하는 방식을 **의사결정트리 (Decision Tree)** 라고 합니다.

일종의 룰베이스(규칙기반) IF..ELIF 임.
단지 **그 룰을 개발자가 정하지 않고 데이터의 패턴을 보고 알고리즘이 스스로 만들어 냄.**

어떤 질문을 할지,어떤 기준으로 데이터를 나눌지는 과거 데이터 속 패턴을 바탕으로 알고리즘이 스스로 학습합니다. 그래서 결정트리는 예측 성능뿐만 아니라, “왜 이런 판단이 나왔는지”를 설명할 수 있는 모델로 실무에서 자주 활용되는 편 입니다.

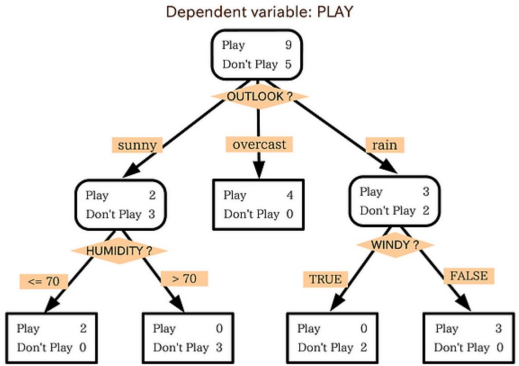
# Decision Tree

## 의사결정나무 알고리즘 (규칙기반 분류 알고리즘)

- 고객 행동 데이터를 if–then 형태의 규칙으로 분해하여 “왜 이런 분류 결과가 나왔는지”를 사람이 이해할 수 있는 구조로 표현

### 알고리즘

스무고개 방식처럼 데이터 특성을 기반으로 질문을 던져 데이터를 분류(Classification)하거나 예측(Regression)하는 지도학습 알고리즘
데이터를 분할할 때 불순도(Impurity)가 낮아지는 방향. 즉, 지니계수(Gini Index)나 엔트로피(Entropy)가 낮아지는(순도가 높아지는) 방향으로 분기




**"밖에 비가 오는 날(rain), 바람이 불지 않으면(windy False) 논다 (Play)."**

# Decision Tree

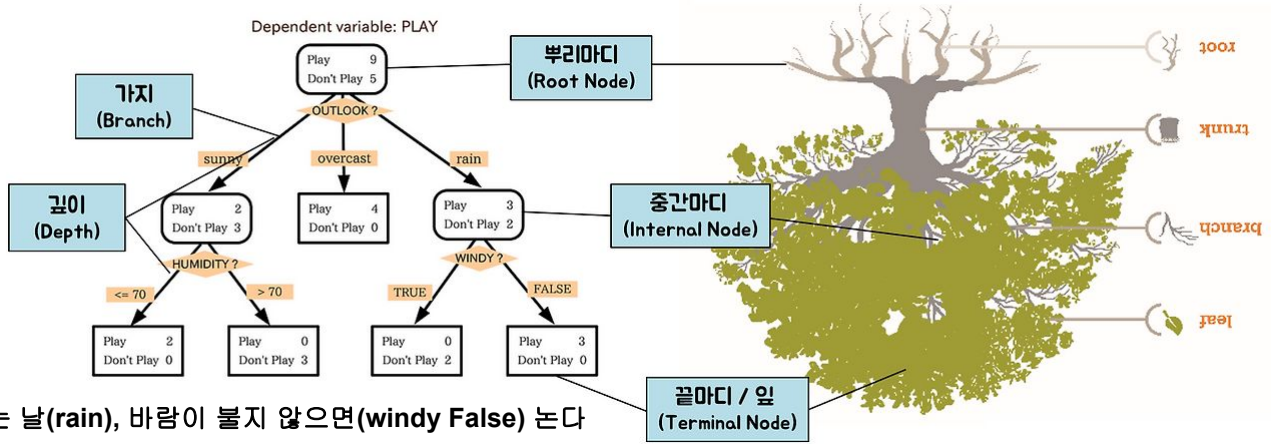
## 의사결정나무 알고리즘 (규칙기반 분류 알고리즘)

- 고객 행동 데이터를 if–then 형태의 규칙으로 분해하여 “왜 이런 분류 결과가 나왔는지”를 사람이 이해할 수 있는 구조로 표현

### 알고리즘

스무고개 방식처럼 데이터 특성을 기반으로 질문을 던져 데이터를 분류(Classification)하거나 예측(Regression)하는 지도학습 알고리즘

데이터를 분할할 때 불순도(Impurity)가 낮아지는 방향. 즉, 지니계수(Gini Index)나 엔트로피(Entropy)가 낮아지는(순도가 높아지는) 방향으로 분기

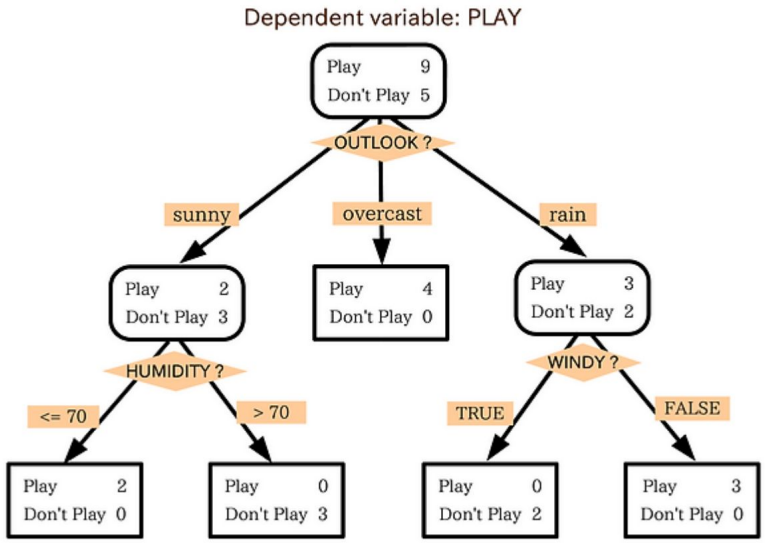



**"밖에 비가 오는 날(rain), 바람이 불지 않으면(windy False) 논다 (Play)."**

# Decision Tree

독립변수 / 타겟변수

| OUTLOOK | HUMIDITY | WINDY | ··· | PLAY |
|---|---|---|---|---|
| Sunny | 20 | True | ··· | Play |
| Sunny | 31 | True | ··· | Play |
| Sunny | 78 | False | ··· | Don't Play |
| Sunny | 71 | True | ··· | Don't Play |
| Sunny | 90 | False | ··· | Don't Play |
| Overcast | 50 | False | ··· | Play |
| Overcast | 60 | True | ··· | Play |
| Overcast | 90 | True | ··· | Play |
| Overcast | 10 | False | | Play |
| Rain | 100 | False | ··· | Play |
| Rain | 90 | False | ··· | Play |
| Rain | 87 | True | ··· | Don't Play |
| Rain | 89 | True | ··· | Don't Play |
| Rain | 95 | False | ··· | Play |




**"밖에 비가 오는 날(rain), 바람이 불지 않으면(windy False) 논다 (Play)."**

# Decision Tree ( CART 알고리즘 )

**CART(Classification And Regression Trees)** : 분류 및 회귀 알고리즘에 모두 사용가능

**지니계수 :** 불순도를 말하는 것으로 예를 들어 데이터넷에서 클래스 0과 1의 비중이 비슷하다면 불순도가 높은 것.

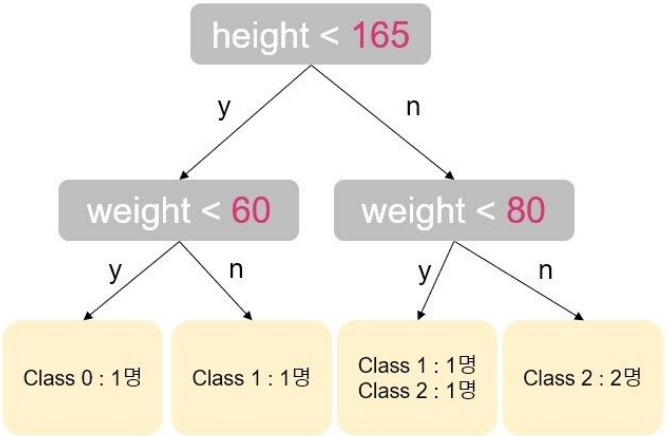
특히, 클래스 0과 1의 개수가 똑같다=정보 차이가 존재하지 않는다는 의미로 어느 클래스에 속할 지 예측할 수 없는 불확실성이 아주 큰 상태이다.

의사결정나무는 불순도, 즉 지니계수가 낮아지는 방향으로 계속 분기를 하는데 분류의 경우 최종적으로는 높은 예측 정확도를 목표로 할 수 있을 것

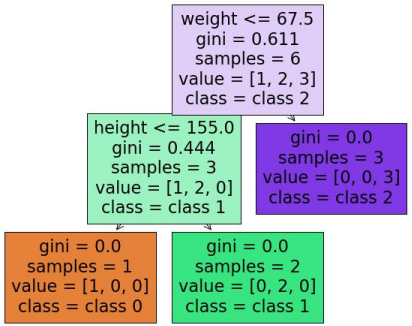
**Decision Tree (CART – Classification and Regression Tree)**

features

| Height | Weight | Class |
|---|---|---|
| 150 | 50 | 0 |
| 160 | 60 | 1 |
| 165 | 65 | 1 |
| 170 | 70 | 2 |
| 180 | 80 | 2 |
| 185 | 85 | 2 |

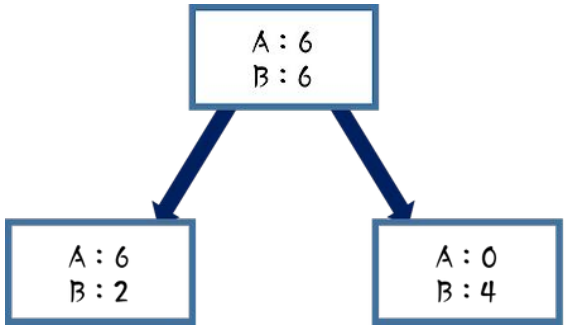



코드 실행 실제 모습



# 지니계수

$$G = 1 - \sum_{i}^{c} p_i^2 \quad (0 \le G \le \frac{1}{2})$$




분리 전: $1 - (1/2)^2 - (1/2)^2 = 1/2$

분리 후: $\{1 - (3/4)^2 - (1/4)^2\} * (2/3) + \{1 - (0/4)^2 - (4/4)^2\} * (1/3) = 1/8$

A와 B가 각각 6개로 구성된 어떤 데이터가 있습니다.
우선 분리가 이뤄지기 전의 지니지수를 계산해봅시다. A의 비율이 1/2이고, B의 비율이 1/2이므로 분리 전의 지니지수는 1/2가 됩니다.
그런데 분리 후의 지니지수 계산 시에는 고려해야 하는 사항이 하나 더 있습니다. 바로 원래의 마디로부터 각 마디로 얼마만큼의 데이터가 할당되었느냐입니다.

뿌리노드를 통해 원래 데이터가 총 12개라는 것을 알 수 있습니다. 그리고 12개의 데이터는 분리 후 왼쪽 마디에 8개, 오른쪽 마디에 4개가 할당되었습니다.
즉, 왼쪽 마디에는 전체의 2/3의 데이터가, 오른쪽 마디에는 전체의 1/3의 데이터가 할당된 것입니다. 이 값을 각각의 마디에 대한 지니지수 계산 후 곱해주어야 합니다. 위의 예시에 적혀있는 계산식처럼 말이죠!

# 시나리오 #9

결정트리 모델을 도입한 이후,마케팅 팀은 고객 이탈 패턴을 규칙 형태로 직접 확인할 수 있게 되었습니다.
하지만 시간이 지나며 또 다른 문제가 드러나기 시작했습니다.

고객 데이터가 늘어나자 결정트리는 점점 더 많은 질문을 만들기 시작했습니다.

가입기간이 14개월 이상인가?
최근 7일 중 평일 접속이 3회 이상인가?
쿠폰 사용 후 구매까지 걸린 시간이 18분 이하인가?

질문은 점점 세밀해졌고, 나무는 끝없이 가지를 뻗어 나갔습니다.그 결과, 과거 데이터에는 매우 잘 맞지만 새로운 고객에게는 예측이 불안정한 모델이 되어버렸습니다.

즉, 결정트리가 학습 데이터의 특이한 패턴까지 모두 외워버린 상태가 되어 과거 데이터에만 너무 잘 맞는 규칙을 만들게 되어 새로운 데이터를 분류하도록 하면 잘 못 분류하는 문제가 발생하게 됩니다. 이렇게 학습에 사용된 데이터셋만 잘 분류하는 상황을 과적합 (overfitting) 되었다고 합니다.

# 시나리오 #9

**새로운 접근: 한 그루 말고, 여러 그루**
머신러닝 팀은 생각을 바꿉니다.

“하나의 완벽한 규칙을 만들려고 하지 말고, 여러 개의 ‘조금씩 다른 판단’을 모아보면 어떨까?”

고객 데이터를 조금씩 다르게 뽑고, 질문 기준도 조금씩 다르게 만든 여러 개의 결정트리를 학습시킵니다.
그리고 각 트리가 내린 판단을 **투표(voting)** 로 모아 최종 결정을 내리기로 합니다.

이렇게 여러 개의 결정트리를 숲처럼 묶어 하나의 결과를 만드는 방법을 Random Forest 라고 합니다.
각각의 트리는 완벽하지는 않지만, 어떤 트리는 특정 고객군에 강하고 어떤 트리는 다른 행동 패턴에 강합니다.

이 다양한 관점을 모으면서 특정 데이터에만 과하게 맞춰진 규칙은 줄어들고, 전체적으로 안정적인 분류 성능을 얻게 됩니다.

**“하나의 복잡한 알고리즘보다 여러개의 간단한 모델이 더 강력할 수 있다는 아이디어”**

여러모델의 결과를 결합하여 보다 정확한 분류/예측 결과를 추출하는 방법을 **앙상블기법** 이라고 하며 랜덤포레스트는 대표적인 앙상블기법의 알고리즘.

# 시나리오 #9

**각각의 결과를 결합하는 방법 2가지**

**1. Voting(투표),**
여러종류의 다른 분류모델을 사용.
같은 데이터를 활용하여 여러 종류의 모델에 적용
각각의 모델에 대한 산출 결과 중에서 가장 많이 나온 결과를 채택

**2. Bagging(bag에 넣기, 포장) or (Bootstrap Aggregating 을 줄여서 만든용어)**
Bagging(배깅)은 하나의 모델(다른 모델을 써도 됨)을 다양하게 학습.
전체 데이터에서 무작위로 데이터를 재추출(부트스트랩)하여 여러개의 데이터셋 샘플을 구성하여 다른 데이터를 학습

아이디어는 모집단의 성질에 대해 표본을 통해 추정할 수 있는 것처럼, 표본의 성질에 대해서도 재표집(resampling)을 통해 추정할 수 있다는 것이다. 즉 주어진 표본(샘플)에 대해서, 그 샘플에서 또 다시 샘플(재표본)을 여러번(1,000~10,000번, 혹은 그 이상)추출하여 표본의 평균이나 분산 등이 어떤 분포를 가지는가를 알아낼 수 있다.

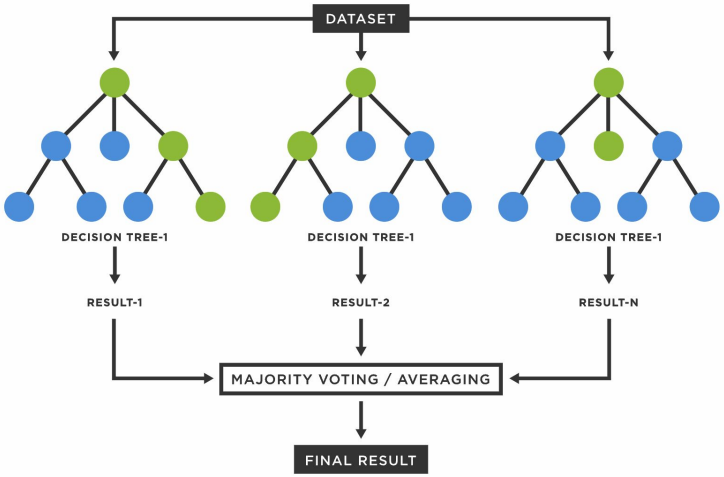
# Random Forest

## 랜덤 포레스트 알고리즘 (앙상블)

- 결정트리를 조금 더 보강하는 알고리즘 [ 의사결정나무 + 배깅 ]
- 여러 개의 서로 다른 결정트리를 결합하여 과적합을 줄이고 더 안정적이고 정확한 고객 분류 수행

### 알고리즘

여러 개의 의사결정나무(Decision Tree)를 무작위로 생성하고 이들의 결과를 종합(투표/평균)해 예측하는 고성능 앙상블 학습 알고리즘
부트스트랩 샘플링(중복 허용)과 특성 무작위 선택을 통해 트리의 다양성을 확
단일 모델의 과적합(Overfitting) 문제를 해결하고 안정적인 예측 성능을 제공



# 비지도 학습 Unsupervised Learning

- 라벨(정답)이 없는 데이터를 기계가 스스로 학습하여 내재된 구조패턴관계를 찾아내는 머신러닝 방식

## 알고리즘

1. 군집 알고리즘 - K-평균 means
2. 주성분 분석 - PCA

# 시나리오 #10

AI 시스템을 도입한 온라인 커머스 회사의 효율적인 마케팅 운영을 옆에서 지켜본 한 회사가 본인들의 서비스에도 AI 시스템을 도입하기로 결정합니다.
이 회사는 웹과 모바일에서 사용할 수 있는 **자기관리·생산성 관리 서비스**를 운영하고 있다.

사용자는 할 일(To-do)을 등록하고, 일정 관리를 하고, 집중 타이머를 사용하며, 목표 달성 현황을 확인하고 서로 공유 하며 함께 자기관리를 할수 있습니다.
요즘 유행하는 런닝 크루처럼 함께하면 즐겁게 목표를 달성할 수 있지 않을까 하는 아이디어 입니다.

할 일 관리, 일정 관리, 집중 타이머, 목표 공유까지 다양한 기능 덕분에 “혼자보다 함께 관리한다”는 컨셉은 시장에서 좋은 반응을 얻었습니다.

하지만 서비스가 성장하면서 새로운 문제가 하나 드러나기 시작했습니다.
같은 서비스를 쓰고 있지만 사용자들의 사용 방식은 너무나 달랐습니다.

- 어떤 사용자는 매일 To-do를 꼼꼼하게 작성하고 완료율도 매우 높았습니다.
- 어떤 사용자는 목표만 세워두고 일정이나 To-do는 거의 사용하지 않았습니다.
- 어떤 사용자는 집중 타이머만 간헐적으로 사용하며 커뮤니티 기능에는 전혀 참여하지 않았습니다.

문제는, 사용자 유형은 다른데, 서비스 대응은 모두 같았다는 겁니다.
열심히 쓰는 사용자에게도, 거의 안 쓰는 사용자에게도, 특정 기능만 쓰는 사용자에게도,
모두에게 같은 알림, 같은 기능 홍보, 같은 기능 안내, 같은 온보딩 화면을 제공하고 있었습니다.

# 시나리오 #10

기획팀은 사용자 유형을 나누고 싶었지만 막상 기준을 정하려니 쉽지 않았습니다.
“To-do를 5개 이상 쓰면 열성 사용자 일까?”
“접속을 매일 하면 좋은 사용자 일까?”
“집중 타이머를 많이 쓰는 사람이 핵심 일까?”

이런 기준들은 사람이 임의로 정한 규칙일 뿐, 데이터가 말해주는 기준은 아니었습니다.
그리고 가장 큰 문제는, “정답 라벨이 존재하지 않는다는 것.”

**이 서비스에는 “이 사용자는 A유형이다”라는 정답 데이터가 없었습니다.**

그래서 팀은 생각을 바꿉니다.
“정답을 먼저 정하지 말고, 사용자들의 실제 사용 습관을 기준으로 비슷한 사람들끼리 자연스럽게 묶어보자.”

이때 필요한 것이 **비지도 학습(Unsupervised Learning)** 입니다.
비지도 학습은 미리 정해진 정답 없이데이터 속 구조와 패턴을 찾아 사용자 유형을 스스로 발견하는 방법입니다.

# 시나리오 #10

“계획형 사용자”
“집중 도구 중심 사용자”
“가볍게 체크하는 사용자”

데이터에서 패턴이 비슷한 사람들끼리 묶어 몇 개의 그룹(클러스터)으로 나눕니다. 이렇게 묶는 과정을 군집화 Clustering 이라고 부릅니다.
이 군집화 알고리즘 중에 가장 대표적인 알고리즘이 **K-Means** 입니다.

# 군집화 Clustering [ K Means ]

## 군집화 알고리즘 (K Means 알고리즘 : K 평균)

- “사용 습관 기반 사용자 유형 분석” (To-do / 일정 / 집중시간 / 목표관리 서비스)
- 유사도(유클리디안 거리를 통해 클러스터링) 를 통한 그룹핑

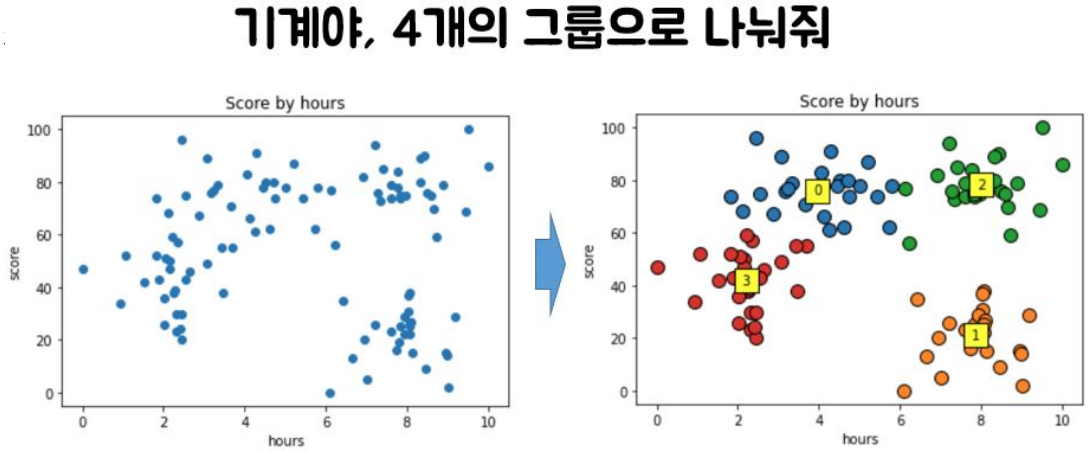
**알고리즘**

라벨이 없는 데이터에서 유사한 특성을 가진 개체끼리 그룹화하는 비지도 학습 기법으로데이터 간 거리(유사도)를 기반으로 군집 내 유사성은 최대화하고 군집 간 유사성은 최소화

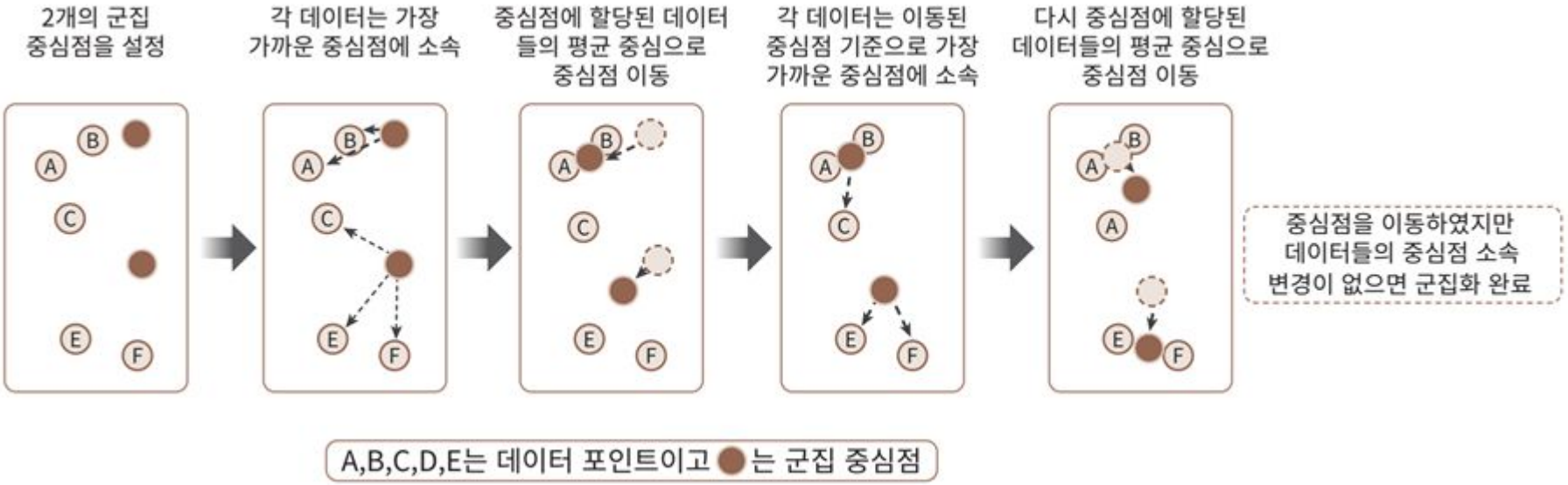
임의의 중심점(centroid) 로부터의 거리를 통해 그룹

유사도 거리계산(유클리디안 거리)

$$d(A,B) = \sqrt{(x2-x1)^2 + (y2-y1)^2}$$



# 군집화 Clustering [ K Means ]




STEP 1 : 군집의 개수(K) 만큼 임의의 위치에 중심점을 정한다.

STEP 2 : 각 데이터를 가장 가까운 중심점에 해당하는 군집에 소속시킨다.

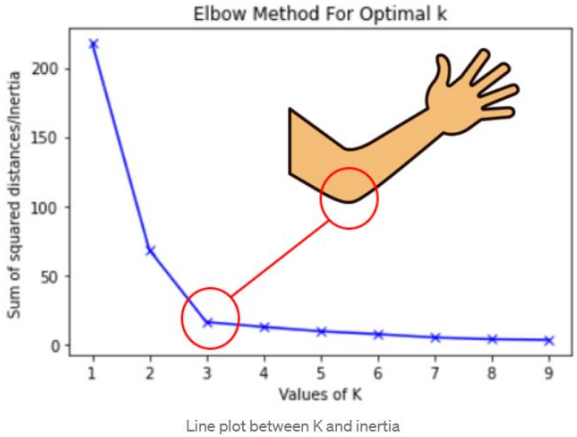
STEP 3 : 각 군집에 속해있는 데이터들의 평균 거리를 갖는 중심위치로 군집의 중심점을 이동시킨다.

주의 : 초기값에 민감

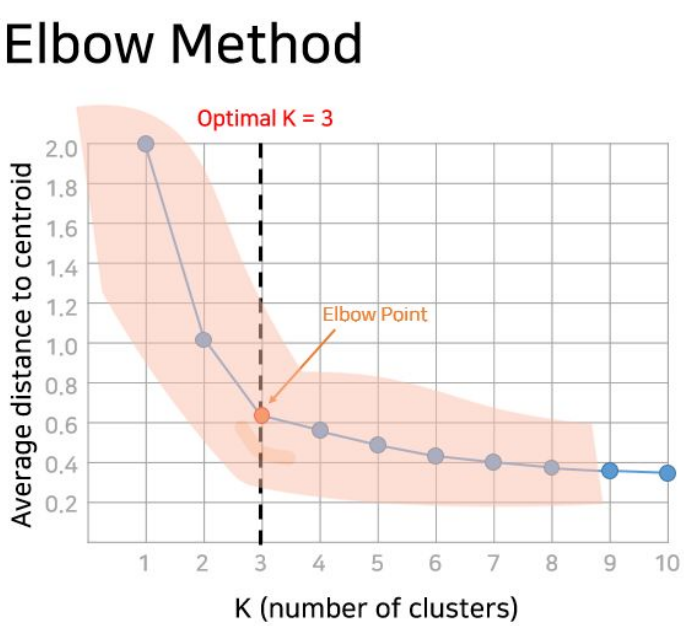
개선 방식
**K-Means ++** 등장

# 최적의 K 개를 찾는 방법 Elbow method

**Elbow 메소드**
클러스터 수 k를 변화시키면서 각 k에 대한 비용함수를 계산하며 최적의 k를 선정하는 방법
일반적으로 비용함수는 SSE(sum of Squared Error)를 사용하며 군집 수 가 증가함에 따라(즉 k가 증가함에 따라) SSE가 급격히 감소하다가 완만해지는 팔꿈치 지점을 찾아 최적의 클러스터 k 수를 찾게 됨



Line plot between K and inertia



# 시나리오 #11

클러스터링 결과는 나왔지만, 기획팀은 다시 질문합니다.

“이 사용자는 왜 이 그룹에 들어간 거죠?”
“변수가 너무 많아서 한눈에 이해하기가 어렵습니다.”

features
주간 평균 로그인 일수, To-do 생성 개수, To-do 완료율, 평균 접속 시간, 집중 타이머 사용 횟수, 목표 설정 대비 달성률, 커뮤니티 참여 횟수

사용자 행동을 설명하는 지표는 10개, 20개로 계속 늘어나고 있었기 때문입니다.

머신러닝팀은 이 문제를 해결하기 위해 팀은 주성분 분석(PCA) 을 도입합니다.
PCA는 여러 개의 사용 지표를 가장 중요한 몇 가지 축으로 요약하는 방법입니다.

예를 들어,
로그인 횟수, 접속 시간, To-do 수 → “활동량”
완료율, 목표 달성률 → “실행력”

처럼 **비슷한 정보를 담은 지표들을 하나의 대표적인 방향으로 압축**합니다.

# 주성분 분석 PCA

## 주성분 분석 PCA 알고리즘 (Principal Component Analysis) - 차원축소

- 복잡한 사용자 행동을 핵심 패턴으로 요약하기

**알고리즘**

기존 feature들을 선형 결합해서 분산(variance)을 가장 크게 설명하는 새로운 축(주성분)을 만듦
( PC1 = 0.6*x1 + 0.3*x2 - 0.7*x3)
( PC2 = ….. )

결과 feature는 원래 feature와 1:1 대응하지 않음
feature 수가 많고 서로 강하게 상관되어 있고 “설명력 유지 + 단순화”가 필요할 때 사용

**PCA는 “이 데이터에서 제일 중요한 기준이 뭔지”를 말로 알려주는 게 아니라, 숫자로 된 방향표만 준다.
그 방향을 보고 사람이 의미를 붙인다.”**

| | PC1 | PC2 | PC3 | PC4 | PC5 | PC6 |
|---|---|---|---|---|---|---|
| 0 | -3.043476 | 0.646301 | -0.249843 | 0.704499 | -0.066913 | 0.376684 |
| 1 | 1.926976 | 0.528713 | 0.431355 | 1.255031 | 0.085687 | 0.642017 |
| 2 | -1.501383 | -0.117994 | 0.800846 | 0.379732 | -0.367393 | -0.553751 |
| 3 | 1.168100 | -1.373146 | 0.668054 | -0.608071 | 0.036437 | -0.285106 |
| 4 | 0.226542 | -0.077045 | -1.553274 | -1.107927 | -0.463466 | -0.381861 |

# 주성분 분석 PCA 예시

**내 플레이리스트에 노래 3,000곡**
feature: 템포, 에너지, 감성 지수, 가사 밀도, 음량

“노래는 많은데 뭐 듣지?”
“공부할 때 / 운동할 때 느낌 다름”

PCA 연결
사실 중요한 건:
‘잔잔 ↔ 신남’
‘감성 ↔ 파워’

PCA가 이걸 자동으로 축으로 만들어줌
“수많은 기준 → 핵심 분위기 2~3개로 요약”

**원래 feature 공간**
각 노래는 5차원 점:
[템포, 에너지, 감성지수, 가사밀도, 음량]

노래 3000개의 점들이 이 공간에 흩어져 있음.

**PCA 에 대한 오해**
“이 축은 감성입니다”
“이 축은 신남입니다”
이런 건 전혀 안 함

**PCA가 실제로 하는 일**
데이터 전체에서 “값이 가장 많이 흔들리는 방향”
을 찾음
그 방향이 주성분(PC1, PC2, …)
이름 없는 숫자 축만 만들어냄

# 주성분 분석 PCA 예시

**PCA Step 1: "같이 움직이는 애들 찾기"**
실제 데이터에서는 보통 이런 상관관계가 생김:

템포 ↑ → 에너지 ↑, 음량 ↑
감성 지수 ↑ → 템포 ↓, 에너지 ↓
가사 밀도 ↑ → 감성 지수 ↑

즉, 빠르고 시끄러운 노래들끼리 묶이고 느리고 잔잔한 노래들끼리 묶임
PCA는 이걸 상관 구조로 감지함.

**PCA Step 2: "가장 많이 퍼진 방향 하나 뽑기" (PC1)**

PCA가 찾은 PC1이 예를 들면:
PC1 = 0.6*템포 + 0.5*에너지 + 0.4*음량 - 0.3*감성지수 - 0.2*가사밀도

템포, 에너지, 음량 쪽은 같은 방향 ( 더하기 +)
감성, 가사는 반대 방향( 빼기 - )

**변수들의 상관계수 correlation coefficient 을 통해 상관관계 확인가능**

이 축을 따라가면:
한쪽 끝: 빠르고 시끄럽고 에너지 넘침
반대쪽: 느리고 조용하고 감성적

사람이 이 PC1을 보고
"아... 이거 '잔잔 ↔ 신남' 축이네?" 라고 이름을 붙이는 것임.

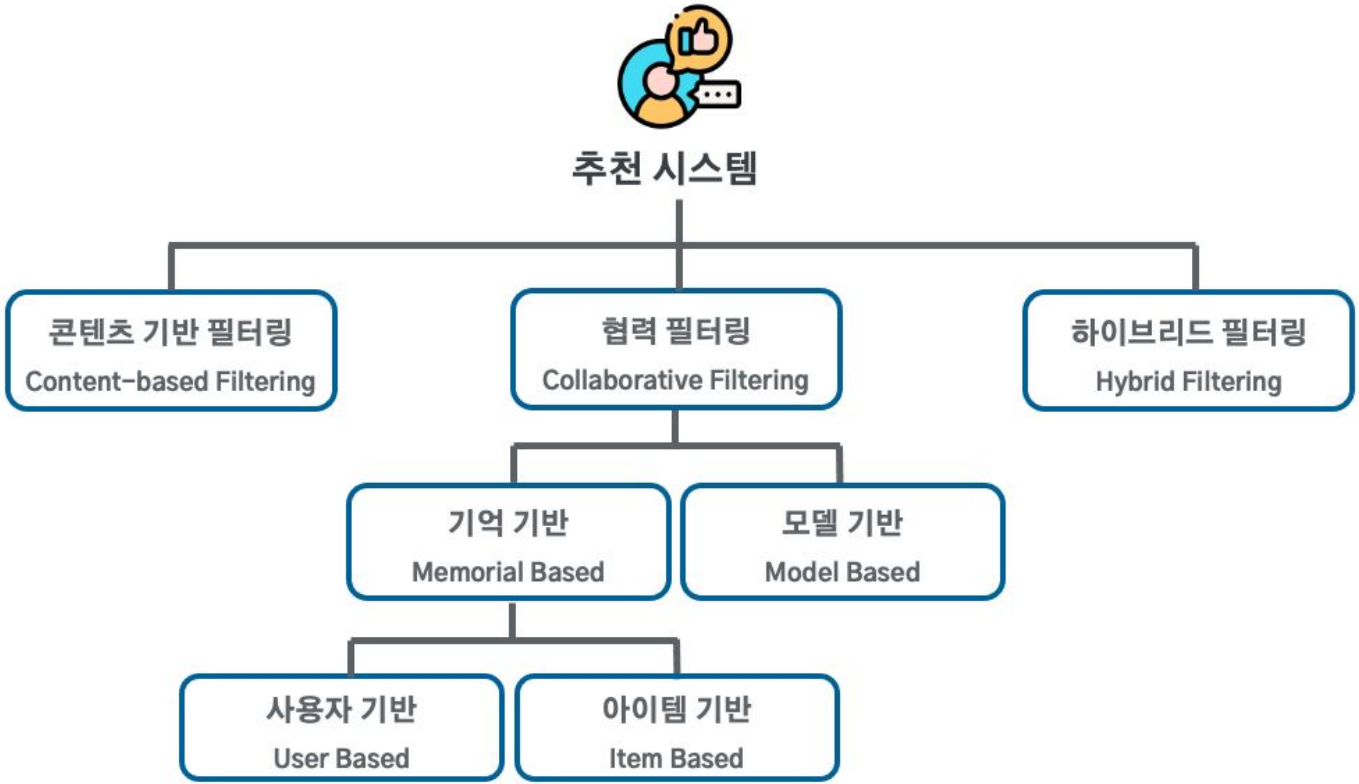
PC2도 마찬가지로 아래와 같다면.
PC2 = 0.7*감성지수 + 0.5*가사밀도 - 0.4*에너지 - 0.3*음량

이 축은:
감성·가사 중심 ↔ 파워·볼륨 중심

그래서 사람이: "이건 '감성 ↔ 파워' 축 같네" 라고 해석함.

**PCA는 "이 데이터에서 제일 중요한 기준이 뭔지"를 말로 알려주는 게 아니라, 숫자로 된 방향표만 준다. 그 방향을 보고 사람이 의미를 붙인다."**

# 파이썬 추천 시스템 Python Recommendation System



# 파이썬 추천 시스템 Python Recommendation System

## 인구통계학적 필터링 (Demographic Filtering)

많은 사람들이 좋아하는 아이템 추천(인기있는 아이템)

**Cold Start** 데이터가 적거나 서비스 초기상태 추천에서 효과적

## 콘텐츠 기반 필터링 (Content-based Filtering)

아이템의 특징(영화: 장르, 배우, 감독 등)을 분석하여 비슷한 특징을 가진 다른 항목을 추천하는 방식

사용 모듈 및 기법 : 자연어처리( Bag Of Word - BOW, TF-IDF 벡터화 ), 코사인 유사도를 통한 콘텐츠의 유사도 판별

## 협업 필터링 (Collaborative Filtering)

나와 비슷한 취향을 가진 다른 사용자들이 좋아한 아이템을 추천하는 방식

사용자의 평점, 구매 이력 등 행동 기반의 데이터를 주로 사용

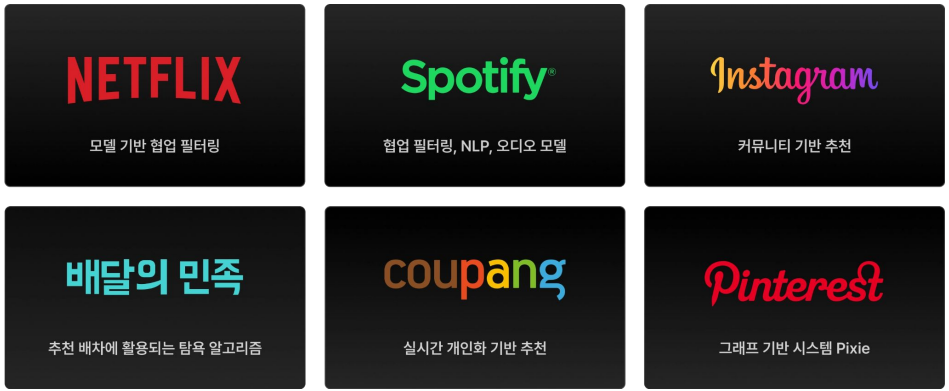
사용 모듈 및 기법 : User based, Item based, 코사인 유사도, scikit-surprise, SVD, Matrix Factorization, 사용자-아이템 행렬(User-Item Matrix)

# 추천 시스템 주요 활용처

**넷플릭스, 유튜브**: 사용자 시청 기록 기반 영화/영상 추천.

**e커머스**: 구매 상품과 유사한 상품 또는 다른 구매자가 구매한 상품 추천.

**소셜 미디어**: Facebook, LinkedIn 등에서 친구 추천.




파이썬 기반 추천 시스템 Python Recommendation System 은

데이터 분석가나 개발자가 머신러닝을 활용해 사용자 맞춤형 서비스를 구현하는 데 있어 가장 필수적이고

기초적인 도구

# IMDB 에서 사용하는 가중치 평점(WR) 공식

대중들이 좋아한다는 것은 평점이 좋아야 하는 것이지만. 사실 평가의 개수가 많기도 해야 함.

**영화1** : 영화 평점이 **10점** 5명이 평가
**영화2** : 영화 평점이 **8점** 500명이 평가

2개의 영화 중에서 어떤 영화의 평점이 조금 더 신뢰있는 것일까? 당연히 개수가 많으면서 평점이 좋은 것이 더 신뢰있음.
그래서 그냥 vote_average 컬룸의 정보만으로 추천하면 안됨. 인기 점수를 계산하기 위해 계산법이 필요함.

**가중치(weight)이 적용된 평점(rating) 계산** [ 이건 추천 서비스를 개발하는 개발자가 고안한 아이디어 ]

$$\text{Weighted Rating (WR)} = \left(\frac{v}{v+m} . R\right) + \left(\frac{m}{v+m} . C\right)$$

- v is the number of votes for the movie;
- m is the minimum votes required to be listed in the chart;
- R is the average rating of the movie; And
- C is the mean vote across the whole report

# 자연어 처리 NLP Natural Language Processing

머신러닝을 이용하면 긴글, 리뷰, 뉴스 같은 자연어를 분석할 수 있음.
이를 **'자연어 처리'** 라고 부름

자연어를 분석한다고 해서 컴퓨터가 글씨를 읽는 것은 아님. 컴퓨터는 글씨를 읽지 못함
그래서 모든 자연어 처리의 전처리 단계는 글씨를 숫자화 시키는 것임.

## 텍스트를 수치화 하는 2가지 방법

**벡터화 Vectorization : (텍스트 -> 숫자)**
텍스트 데이터를 단순히 숫자로 변환하는 모든 기술을 통칭
통계적 방식: 주로 빈도수(Count)나 통계적 중요도(TF-IDF)를 기반
One-Hot Encoding, Bag of Words(BoW), TF-IDF

**임베딩 Embedding : (텍스트 -> 숫자+유사도)**
단어나 문장을 저차원 공간에 밀집된 밀집 벡터(Dense Vector)로 표현하며, 의미적 유사성을 반영하는 기술
신경망 등을 통해 단어의 맥락(문맥)을 학습하여 의미를 수치화
유사한 의미를 가진 단어는 벡터 공간에서 가까운 위치에 배치
Word2Vec, GloVe, FastText, BERT, GPT

# 자연어 처리 벡터화 : BOW

**Bag Of Word**

문자열을 수치값으로 변환하는 벡터화의 대표적 알고리즘인 BOW (단어 빈도 기반)

각 문장에 나온 단어의 개수를 세어 숫자로 만들기
각 문장별로 단어의 출현 빈도를 표시한 값을 feature 벡터화 라고 부름
빈도수 카운트가 높은 단어가 중요한 단어라고 판단 할 수 있음.

별 헤는 밤

윤동주

문장1 : 별 하나에 추억과
문장2 : 별 하나에 사랑과
문장3 : 별 하나에 쓸쓸함과
문장4 : 별 하나에 동경과
문장5 : 별 하나에 시와
문장6 : 별 하나에 어머니, 어머니,




*백오브워즈 단어 빈도수 표1

| 별 | 하나 | 에 | 추억 | 과 | 와 | 사랑 | 쓸쓸함 | 동경 | 시 | 어머니 |
|---|---|---|---|---|---|---|---|---|---|---|
| 6 | 6 | 6 | 1 | 4 | 11 | 1 | 1 | 1 | 1 | 2 |

*백오브워즈 문장별 단어 빈도수 표2

| | 별 | 하나 | 에 | 추억 | 과 | 와 | 사랑 | 쓸쓸함 | 동경 | 시 | 어머니 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 문장1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 문장2 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 |
| 문장3 | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| 문장4 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| 문장5 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 문장6 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 |

# 자연어 처리 벡터화 : TF-IDF

## Term Frequency - Inverse Document Frequency

각 문장안에 단어가 많이 등장하면 중요한 단어지만, 반대로 다른 문서에도 많이 등장하면 중요하지 않다고 보는 알고리즘

대표 예:
**a, am** 와 같은 관사, 전치사 등 글 의미와는 상관없는 중요하지 않은 단어(불용어 **stop word**)지만 모든 문서에 많이 등장하기에 이에 대한 패널치 반영

**핵심 공식:** $TF - IDF(t, d) = TF(t, d) \times IDF(t)$

$$\mathrm{TF}(t, d) = \frac{\text{단어 } t\text{의 등장 횟수}}{\text{문서 } d\text{의 전체 단어 수}}$$

$$\mathrm{IDF}(t) = \log\left(\frac{N}{df(t)}\right)$$

**TF (Term Frequency)** : 단어 t가 특정 문서 d 내에서 등장하는 빈도. 높을수록 해당 문서에서 중요함.

**IDF (Inverse Document Frequency)** : 전체 문서 수(N)를 단어 t가 포함된 문서 수(df)로 나눈 후 로그(log)를 취한 값.

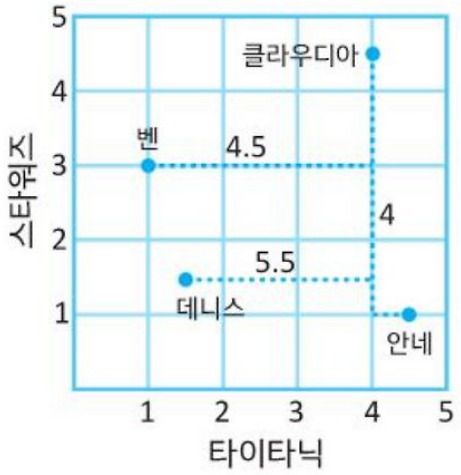
특정 단어가 다른 문서에서 거의 등장하지 않을수록 값이 커짐.

계산방식 예 ) https://wikidocs.net/31698

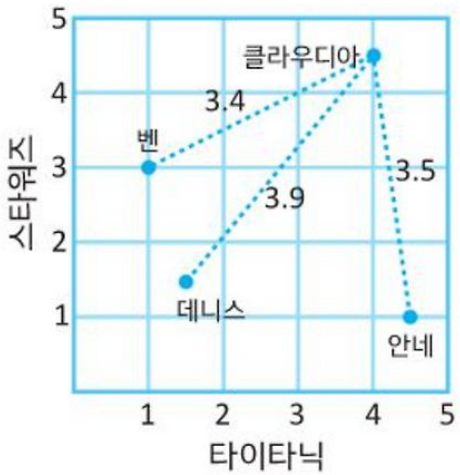
# 유사도 Similarity

| | 타이타닉 | 스타워즈 |
|---|---|---|
| 안네 | 4.5 | 1 |
| 벤 | 1 | 3 |
| 클라우디아 | 4 | 4.5 |
| 데이스 | 1.5 | 1.5 |

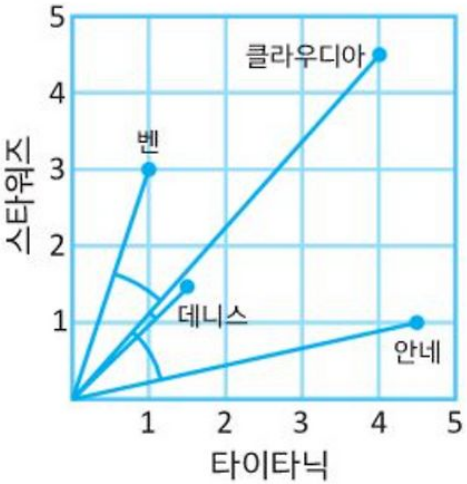
※ 클라우디아 와의 유사도 계산



<Manhattan Distance>



<Euclidean Distance>



<Cosine Similarity>

내적 (A · B)

$$\cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

벡터 크기

# 협업 필터링 Collaborative Filtering

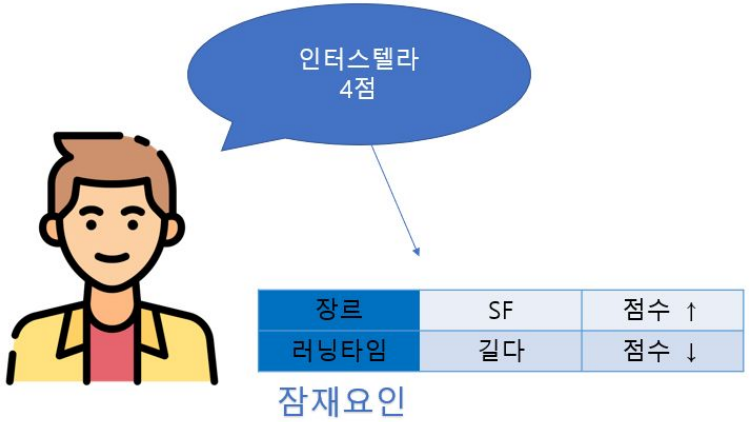
나와 비슷한 취향을 가진 다른 사용자들이 좋아한 아이템을 추천하는 방식 : 사용자의 평점, 구매 이력 등 행동 기반의 데이터를 주로 사용

| 사용자 | 아이템 | 평점 |
|---|---|---|
| 철수 | 인터스텔라 | 4 |
| 철수 | 테넷 | 3 |
| 영희 | 인터스텔라 | 3 |
| 영희 | 어바웃타임 | 2 |
| 준수 | 분노의 질주 | 5 |

| | 인터스텔라 | 어바웃타임 | 테넷 | 분노의 질주 |
|---|---|---|---|---|
| 철수 | 4 | | 3 | |
| 영희 | 3 | 2 | | |
| 준수 | | | | 5 |

**User - Item sparse matrix** 사용자-아이템 희소행렬 (사용자 * 아이템)

값의 대부분이 비어있어서 희소(sparse) 라고 부름




잠재요인 예시) 폭력성 ,선정정, 코믹, 장르, 배우, 러닝타임, 언어, 멜로, 드라마, ...

# 협업 필터링 Collaborative Filtering

user-item (4X5)

| | Item1 | Item2 | Item3 | Item4 | Item5 |
|---|---|---|---|---|---|
| User1 | 4 | | | 2 | |
| User2 | | 5 | | 3 | |
| User3 | | | 3 | 4 | 4 |
| User4 | 5 | 2 | 1 | 2 | |



사용자와 아이템의 잠재벡터 개수 **K**는
반드시 같아야 함

user-잠재요인 (4X2)

| | 요인1 | 요인2 |
|---|---|---|
| User1 | 0.94 | 0.96 |
| User2 | 2.14 | 0.08 |
| User3 | 1.93 | 1.79 |
| User4 | 0.58 | 1.59 |

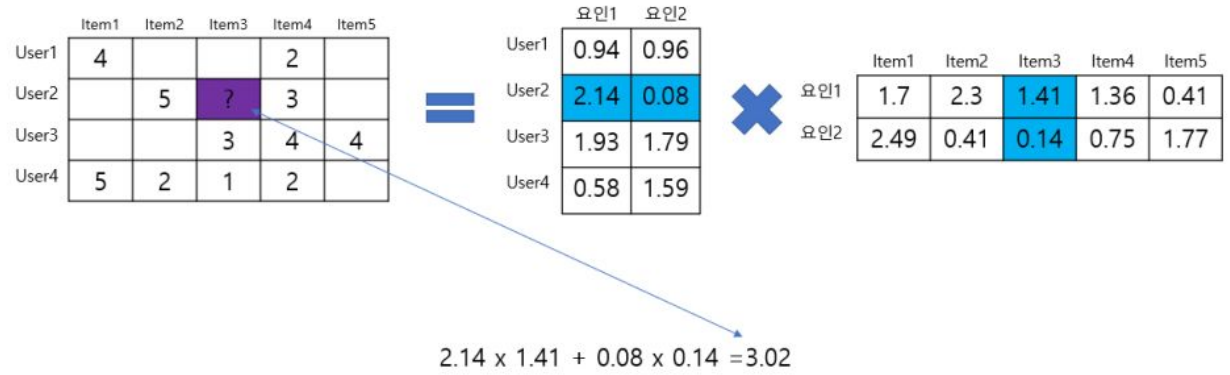


잠재요인-item (2X5)

| | Item1 | Item2 | Item3 | Item4 | Item5 |
|---|---|---|---|---|---|
| 요인1 | 1.7 | 2.3 | 1.41 | 1.36 | 0.41 |
| 요인2 | 2.49 | 0.41 | 0.14 | 0.75 | 1.77 |

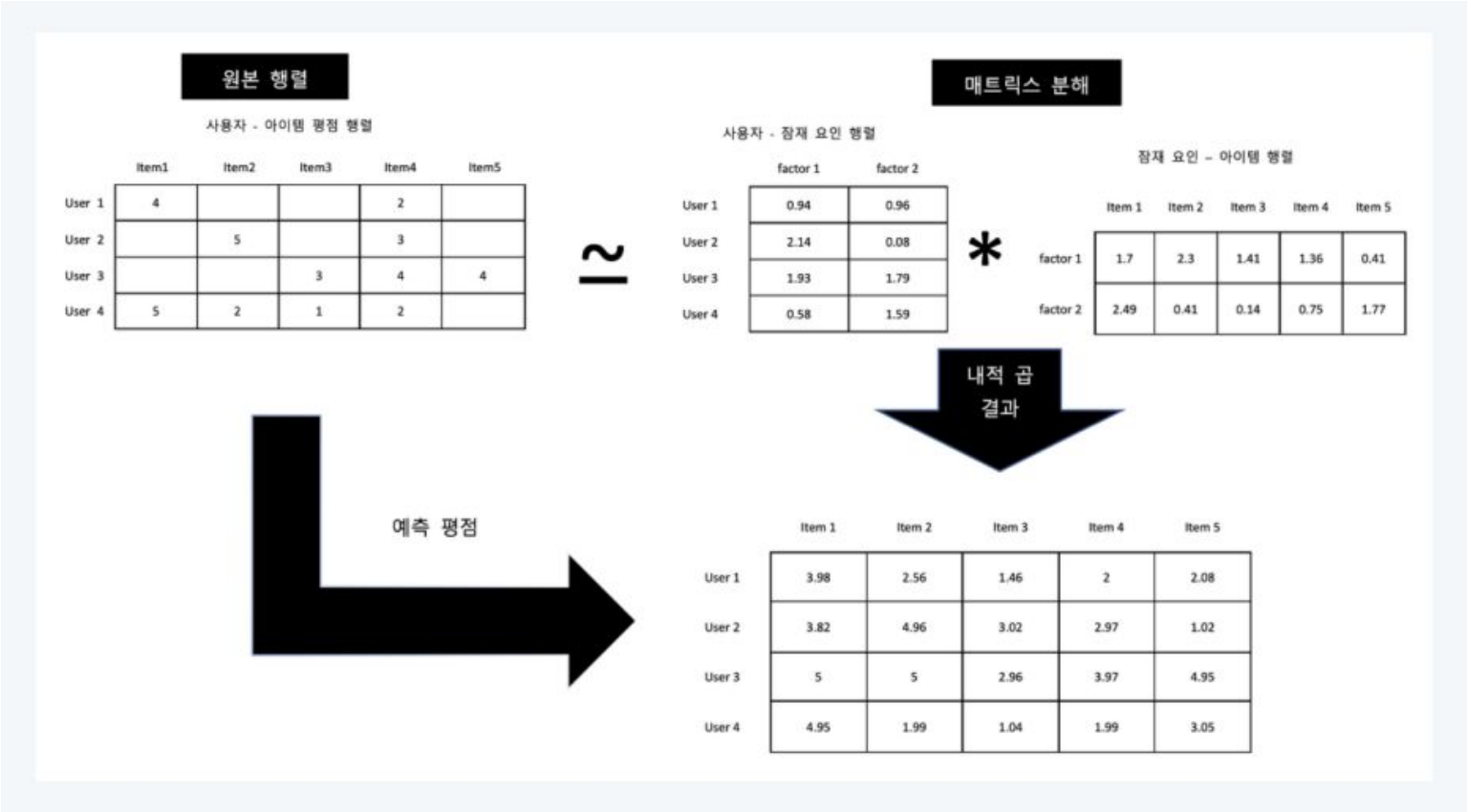
잠재요인 latent factors 벡터값은 **model**이 평점 데이터 학습을 통해 계산한 값. 사람은 각 요인이 어떤 특성인지 알 수 없음.

# 협업 필터링 Collaborative Filtering

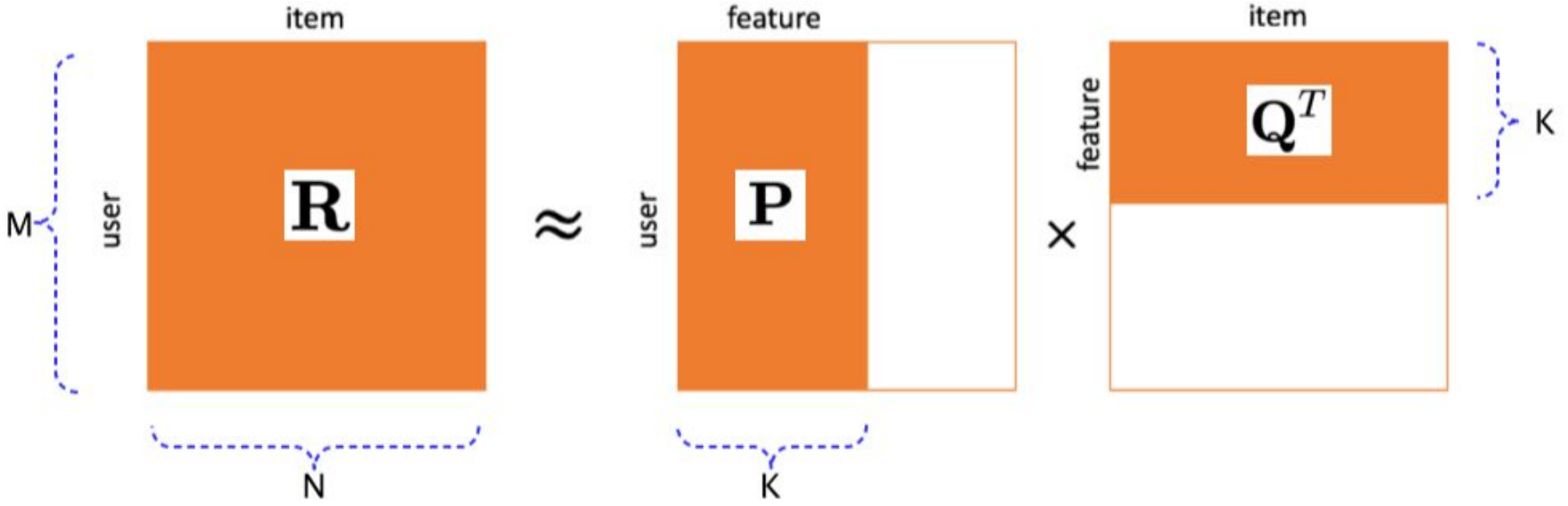





# 협업 필터링 Collaborative Filtering



# MF : Matrix Factorization 행렬 분해




$$\mathbf{R} \approx \mathbf{P} \times \mathbf{Q}^T = \hat{\mathbf{R}}$$

R: Rating matrix  P: User latent matrix  Q: Item latent matrix

# MF : Matrix Factorization 행렬 분해

**Latent Factor를 2개로 설정**

| 사용자 | 액션 – 드라마 ( -1 ~ 1 ) | 판타지 – 사실주의 ( -1 ~ 1 ) |
|---|---|---|
| Bob | -0.43 | 0.21 |
| Sue | 0.31 | 0.92 |
| Mary | 0.69 | -0.03 |
| Alice | 0.46 | -0.30 |

사용자 요인 (P matrix)

| 영화 | 액션 – 드라마 ( -1 ~ 1 ) | 판타지 – 사실주의 ( -1 ~ 1 ) |
|---|---|---|
| 기생충 | 0.31 | 0.60 |
| 겨울왕국 | 0.61 | -0.82 |
| 부산행 | -0.38 | -0.61 |
| 백두산 | -0.79 | 0.08 |

아이템 요인 (Q matrix)

$$\hat{R}_{ui} = P_u \cdot Q_i^T$$

| 영화 / 사용자 | 기생충 | 겨울왕국 | 부산행 | 백두산 |
|---|---|---|---|---|
| Bob | -0.0073 | -0.4345 | 0.0353 | **0.3565** |
| Sue | **0.6481** | -0.5653 | -0.679 | -0.1713 |
| Mary | 0.1959 | **0.4455** | -0.2439 | -0.5475 |
| Alice | -0.0374 | **0.5266** | 0.0082 | -0.3874 |

# MF : Matrix Factorization 행렬 분해

**Latent Factor를 2개로 설정**




**실제 SVD 평점 예측식**

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$$

- μ : 전체 평균
- bu: 사용자 편향
- bi : 영화 편향

# Surprise 모듈

명시적 등급(Explicit Rating) 데이터(예: 1~5점 평점)를 사용하여 추천 시스템을 구축하고 분석하는 데 사용되는 Python 오픈 소스 머신러닝 라이브러리
외부모듈이어서 설치 필요 : !pip install scikit-surprise

### 주요 특징

Surprise 패키지의 풀네임 : "**S**imple **P**ython **R**ecommendation **S**ystem **E**ngine"의 약자로, 이름 그대로 단순하고 사용하기 쉬운 API를 제공

알고리즘 : KNN, Matrix Factorization(SVD, SVD++, NMF) 등 다양하고 전통적인 협업 필터링 알고리즘 바로 사용 가능
평가도구 : 교차 검증(Cross-validation), RMSE, MAE 등 모델 성능을 측정할 수 있는 편리한 도구 제공
데이터셋 : MovieLens 등 내장 데이터셋을 지원하며, 판다스(Pandas) 데이터프레임에서 직접 데이터를 로드할 수 있음

### 핵심 구성 요소

**Reader :** 사용자의 등급 데이터(rating scale)를 파싱하는 데 사용
**Dataset :** 학습용 데이터를 로드하고 관리합니다.
**Prediction Algorithms (SVD, KNN 등) :** 추천을 수행하는 실제 모델

### 동작특징

Surprise는 **사용자 아이디, 아이템 아이디, 평점 데이터**가 로우 레벨로 된 데이터세트만 적용할 수 있음.
그래서 데이터의 **첫번째 컬럼을 사용자 아이디, 두번째 컬럼을 아이템 아이디, 세번째 컬럼을 평점으로 가정해 데이터를 로딩**하고
**네번째 컬럼부터는 로딩을 수행하지 않음**
예로, user_id, item_id, rating, time_stamp 필드로 구분된 데이터라면 앞 3개 필드만 로딩하고 이후 time_stamp 필드는 로딩에서 제외됨.
따라서 반드시 데이터 세트의 컬럼 순서가 사용자 아이디, 아이템 아이디, 평점 순으로 돼 있어야 함.

# Implicit Feedback 기반 추천 시스템

명시적 평점(별점) 이 없고, 시청 내역 / 구매 내역 같은 행동 데이터만 있는 경우

**핵심 아이디어**
"비슷한 행동을 한 사용자끼리는 앞으로도 비슷한 아이템을 좋아할 것이다"

명시적 평점이 없어도
시청 여부 (1 / 0)
구매 여부 (1 / 0) - 개수를 통해 조금 더 강하게 값 부여 가능
클릭 횟수
체류 시간

같은 행동 로그를 선호 신호로 간주해서 학습

**대표 알고리즘**
ALS (Alternating Least Squares for Implicit Data)
BPR (Bayesian Personalized Ranking)

**User - Item 행렬을 만들어서**

| | Item1 | Item2 | Item3 |
|---|---|---|---|
| User1 | 1 | 0 | 1 |
| User2 | 0 | 1 | 1 |

이 **user-item sparse matrix**를 잠재벡터로 분해
U: 사용자 latent vector
V: 아이템 latent vector

Netflix Prize에서 유명해진 방식



# Implicit Feedback 기반 추천 시스템

**명시적 평점(Explicit Rating)**에 사용하는 **SVD**와 **묵시적 평점(Implicit Rating)**에 사용하는 **Implicit ALS** 차이

**공통점:** 잠재벡터(latent factor) “행렬을 쪼개서 취향을 찾는다”
사용자 – 아이템 행렬 → latent factor로 분해 R= Pu * Q(t)i
사용자 벡터 × 아이템 벡터 → 선호도 예측

**차이점**
1. SVD (Explicit Feedback) - 사용 데이터: 별점 (1~5점 등), 값의 의미: 선호도 자체
2. Implicit ALS (Implicit Feedback) - 사용 데이터: 클릭, 구매, 시청 여부 값의 의미: 선호도가 아님. 관심을 보였는지 여부 + 강도.
   1 = '좋아함'이 아님, 1 = 관측됨 (observed interaction)

**Implicit ALS는 다음 두 개로 점수화**
Preference [선호] (p_ui) : 봤으면 1, 안 봤으면 0
Confidence [신뢰](c_ui) : 얼마나 강하게 믿을 것인가, 예: 많이 본 영화 = 높은 confidence

**ALS라는 이름이 붙는 이유 : Alternating Least Squares**
사용자 고정 → 아이템 벡터 업데이트
아이템 고정 → 사용자 벡터 업데이트
번갈아 반복

**score : 유사도를 이용해 계산된 유저-아이템 선호도**
점수 = (내가 본 아이템들) × (아이템 유사도)

$$score(u,i) = \sum_j r_{uj} \cdot sim(j,i)$$